

The background of the entire cover is a blue-toned topographic map. It features intricate contour lines and shaded relief, with a darker blue area in the upper center that resembles a mountain range or a specific geographical feature.

THE NEW/STACK

The Developer's Guide to Cloud Infrastructure, Efficiency and Sustainability

sponsored by

Google Cloud

AMD 

Charles Humble

The New Stack

The Developer's Guide to Cloud Infrastructure, Efficiency and Sustainability

Alex Williams, Founder and Publisher

Ebook Team:

Ben Kubany, Project Manager

Charles Humble, Author

Diana Gonçalves, Graphic Designer

Michael Baker, Copy Editor

Susan Hall, Copy Editor

Vicki Walker, Ebook Editor

Supporting Team:

Andrew Tillery, Senior Marketing Manager

Benjamin Ball, Account Director

Heather Joslyn, Editor-in-Chief

Laila Bailey, Email Marketing Manager

Michelle Maher, Copy Editor

Shelley Chen, Social Media Manager

Tony Sala, Director of Sales

© 2025 The New Stack. All rights reserved.

20250401.01

Table of Contents

Sponsor5

Introduction: What Is Sustainable Software?6

 The Many Meanings of Sustainability in Software6

Why Carbon Emissions Matter8

 Understanding Energy Proportionality 10

IT’s Role in Sustainability 14

 A Strategy for Reducing Carbon Emissions 16

 Mapping Carbon Scopes to IT 18

 Carbon Neutral vs. Carbon Zero20

 How To Achieve an “Inspiring” Rating20

The Developer’s Role in Sustainability22

 Coding Efficiency vs. Operational Efficiency22

 Ways To Improve Operational Efficiency24

 Manual Rightsizing24

 LightSwitchOps27

 Making Full Use of the Cloud28

 Burstable and Preemptible/Spot Instances28

 Serverless First29

Unpacking CPU Performance in the Cloud 31

 Performance Comparisons32

 Matching Workloads to VMs 33

Measuring Carbon Emissions35

 CPU and Hardware Utilization..... 36

 Cloud Costs 36

 Assessment Tools38

 Google’s Carbon Footprint Tool38

 Green Software Foundation Carbon Intensity Formula 44

Taking Steps To Reduce Emissions.....47

 Demand Shifting..... 49

 Graceful Degradation50

 Location-Based Demand Shifting.....50

 Demand Shaping.....52

Putting It All Together.....55

Looking Ahead: Trends and Future Considerations57

 Renewable Energy Certificates.....57

 Carbon Taxation62

 Water Usage62

Final Words 64

 Further Reading66

About the Author67

Disclosure.....68

Sponsor

We are grateful for the support of our sponsors for this ebook:



AMD and Google Cloud collaborate to provide high-performance computing solutions tailored to meet a wide variety of workloads, so you can confidently pursue digital innovation in the cloud.

AMD is the high-performance and adaptive computing leader, powering the products and services that help solve the world's most important challenges. Our technologies advance the future of the data center, embedded, gaming, and PC markets.

Google Cloud helps organizations build quickly, securely, and cost-effectively with infrastructure that is optimized for the demands of AI, modern, enterprise, and distributed workloads.



INTRODUCTION

What Is Sustainable Software?

The Many Meanings of Sustainability in Software

This ebook focuses primarily on energy efficiency as a way to reduce the carbon dioxide emissions resulting from the software we build and run.

This practice is sometimes referred to as building “sustainable software,” but the terms “sustainable” and “sustainability” can be confusing in a software context as they have multiple distinct meanings.

These meanings include:

- The word “sustainable” is used by the agile community to mean that a software team can continue delivering software indefinitely at a given level of velocity.
- There is an academic field of study called “software sustainability,” which is extremely broad, and as some leading researchers described in the [Karlskrona Manifesto](#), encompasses “social, individual, environmental, economic, and technical” aspects of sustainability.
- “Sustainability” also refers to efforts to reduce the environmental impact of software development and operation, sometimes also known as “green software.”

This book explores the third meaning of the term. If you already have a strong understanding of carbon emissions, energy proportionality, and carbon scopes and are here to learn how developers can influence sustainability, feel free to jump to Chapter 3, “[The Developer’s Role in Sustainability](#),” and Chapter 4, “[Unpacking CPU Performance in the Cloud](#).” If not, Chapters 1 and 2 (“[Why Carbon Emissions Matter](#)” and “[IT’s Role in Sustainability](#)”) will give you a solid, if brief, overview of these topics.

As an industry, two major things we need to be concerned with when it comes to environmental impact are carbon dioxide emissions and water consumption from water-based cooling. As software developers, we have much more direct influence on carbon dioxide (hereafter “carbon”), so that will be the focus of this book.



CHAPTER 01

Why Carbon Emissions Matter

Carbon is one of a family of gases known as greenhouse gases (GHGs). These gases prevent the sun's heat from leaving the Earth's atmosphere.

In moderation, GHGs are vital, since without them the Earth would be, [at around -20°C](#) (-4°F), too cold to sustain most life. However, since the Industrial Revolution began in the 1760s, we've been emitting too many GHGs and, as the famous hockey stick graph shows, this has created global warming.

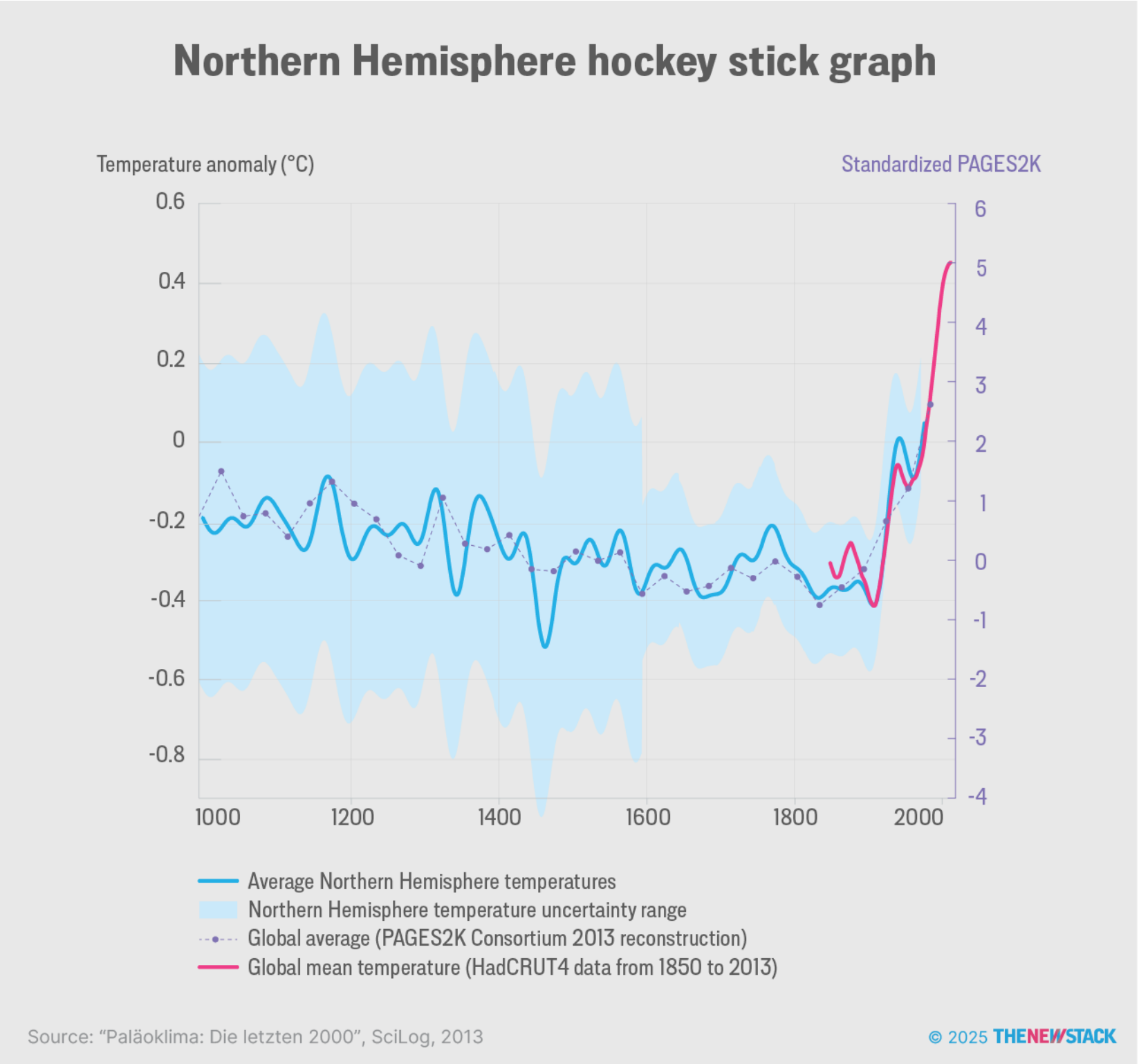


FIG 1.1 The global mean temperature has risen sharply since the Industrial Revolution.

While the +0.6°C degrees of warming shown in this graph may not sound very alarming, the scientific consensus is that an increase of 1.5°C is likely very bad.

With the climate policies we currently have in place, we’re heading toward a

projected [global temperature increase of 2.7°C](#) by the year 2100. We need to do better, but it’s worth saying that if we hadn’t already put some measures in place, we’d be looking at 4°C to 5°C of warming. In other words, we’ve already made real progress.

Understanding Energy Proportionality

The biggest single emitter of carbon dioxide is the burning of fossil fuels such as coal, oil, and natural gas to produce electricity — something we use a lot of in IT. What's more, a large amount of that power is wasted. This is partly something of a historical accident.

Before the public cloud existed, enterprise applications ran in data centers owned and managed by a company's IT team. Procurement cycles were slow, often measured in weeks or even months. A side effect of this was that, if you were tasked with the business of sizing a new application, you would make a point of overprovisioning it as a risk mitigation strategy — perhaps adding an extra server or two to be on the safe side. Those servers sat idle for the most part, but it was extremely unusual for

someone to audit them or requisition them for other uses once an application was in production.

This matters because a server sitting idle uses a surprising amount of power. [Energy-proportional computing](#) is a concept first put forward in 2017 by Google engineers to help evaluate how efficiently a computer converts electricity to practical work.

Google's observation is that most hardware components have a nonlinear relationship between the energy they consume and the work they perform. In other words, hardware becomes more efficient at converting electricity to practical computer operations the more you run on it. The sweet spot will vary depending on the efficiency of the servers being used, but it is typically somewhere around 50% to 80%.

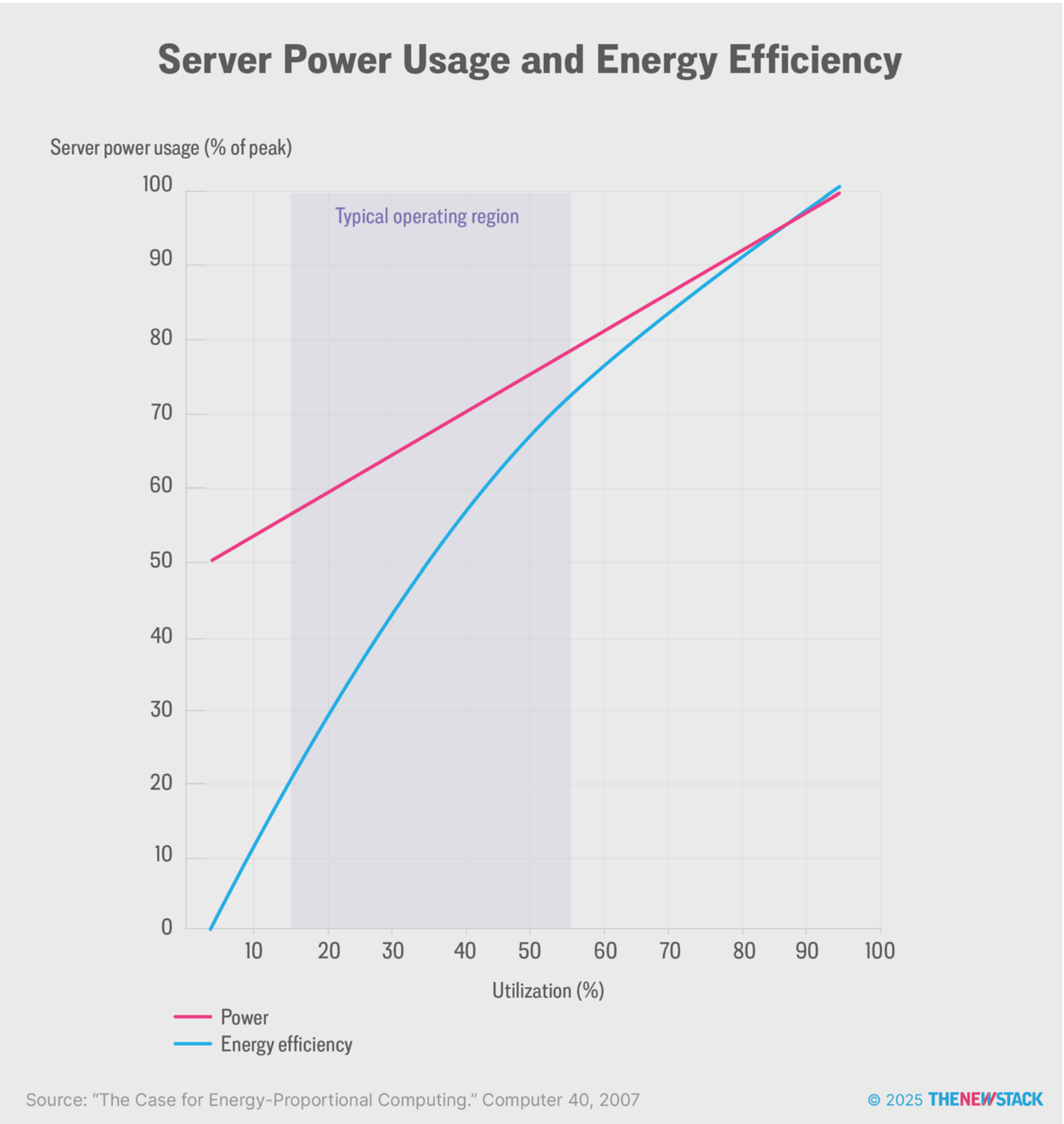


FIG 1.2 Server power usage and energy efficiency at varying utilization levels, from idle to peak performance. Even an energy-efficient server still consumes about half its full power when doing virtually no work. (Source: Barroso, Luiz André, and Urs Hölzle. 2007a. [“The Case for Energy-Proportional Computing.”](#) Computer 40 [12]: 33–37.)

[I've argued elsewhere](#) that it is possible to build green software that resides on premises, but it is also extremely challenging. One reason is that public cloud providers are better able to keep servers running at optimum utilization with techniques such as multitenancy. They're also able to employ large teams of expensive specialists to get their data centers running in the most efficient way possible.

[Google's analysis](#) of its more efficient servers, power infrastructure, and cooling systems suggests that a data center that is owned and operated by Google is, on average, 1.8 times as energy efficient as a typical enterprise data center, compared with data center industry averages, based on 2023 data.

This, plus a variety of other factors, means that you typically want to start

any green software initiative by moving as many workloads as possible to the public cloud.

“Sustainability should join cost, performance, security, regulatory concerns, and reliability as one of the top-level considerations when optimizing your cloud workloads.

However, moving to the cloud is only *potentially* more carbon efficient. It heavily depends on a wide range of factors, including your choice of regions, server density, and CPU, and your use of managed services and spot instances. Of course, you can still overprovision, even in the cloud.

In other words, the cloud can be a lot more sustainable than on-premises infrastructure — but this isn't guaranteed, and simply "lifting and shifting" your existing apps won't cut it. The choices you make as a developer matter. What's more, some of those choices are subtle and may go against the grain. That subtlety is one of the reasons why taking a green approach to building software is such enjoyable, challenging work.

My belief is that sustainability should join cost, performance, security, regulatory concerns, and reliability as one of the top-level considerations when optimizing your cloud workloads. My goal with this book is to provide you with the foundation you need to start lowering the cost and environmental impact of the software your business runs.

At the end, I'll provide recommended resources for you to continue to explore this fascinating and fast-moving area of software architecture.

CHAPTER 02

IT's Role in Sustainability

It may surprise you to learn that energy consumption from data centers has grown only modestly in recent years. Nevertheless, the International Energy Agency (IEA) suggests that estimated global data center electricity consumption in 2022 was [240 TWh to 340 TWh](#), accounting for around 1% to 1.3% of all global electricity demand. That figure excludes the energy used for cryptocurrency mining, which was around 110 TWh in 2022, a further 0.4% of annual global electricity demand.

It also excludes data transmission networks, without which your data center is essentially useless. Data transmission consumes an estimated 260 TWh to 360 TWh, or another 1% to 1.5%, of global electricity use. This more or less doubles the data center electricity consumption figure; that upper estimate is 700 TWh (excluding cryptocurrency). For perspective, this is [similar to Brazil's electricity demand, and more than double the UK's](#). Crypto alone uses about the same amount of power as Norway.

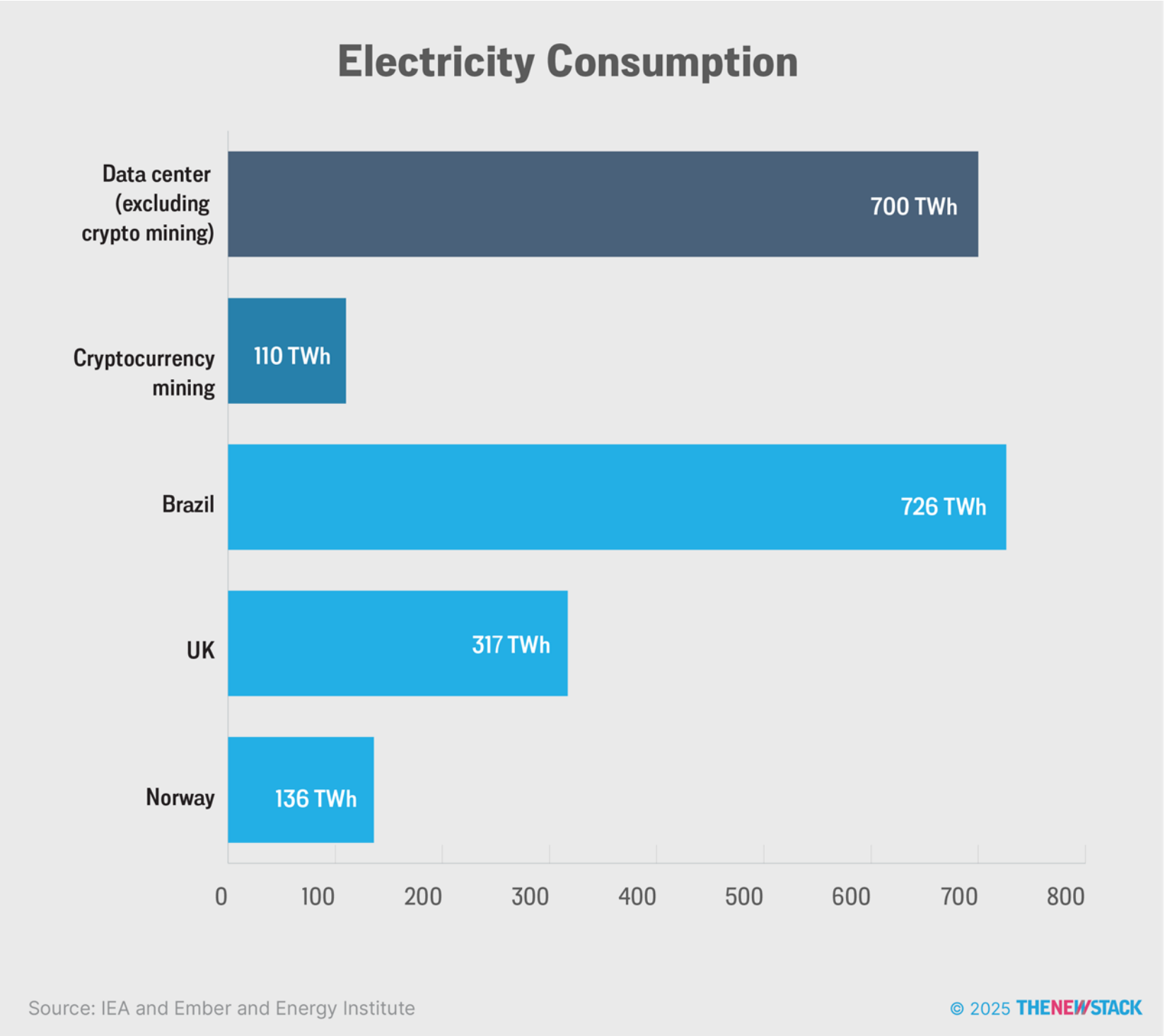


FIG 2.1 Data centers consume a similar amount of electricity as Brazil does. (Sources: [IEA](#) and [Ember and Energy Institute](#))

What’s more, electricity usage is climbing rapidly among the larger tech companies. The IEA notes that the “combined electricity use by Amazon, Microsoft, Google, and Meta more than doubled between 2017 and 2021, rising to around 72 TWh in 2021.”

If we want to reduce the potential impact, we need to think about two major things: reducing the amount of electricity we use as much as possible, and using carbon-free sources of energy whenever we can. To do that, we need a strategy.

A Strategy for Reducing Carbon Emissions

The [Green Software Foundation](#), a nonprofit formed under the Linux Foundation, has produced an open source [Green Software Maturity Matrix](#), which offers a useful starting point. It aims to provide answers to four questions:

1. Where do I start?
2. Where am I now?
3. What am I heading toward?
4. How do I know I am making progress?

Organizations rate themselves from 1 to 5 (corresponding to five levels: aspiring, aware, acting, awesome, and inspiring) in nine dimensions. The goal is to move from the left to the right; in this case, toward “inspiring.” If you take a few minutes to mark roughly where you think your organization is, you’ll perhaps end up with something like this:

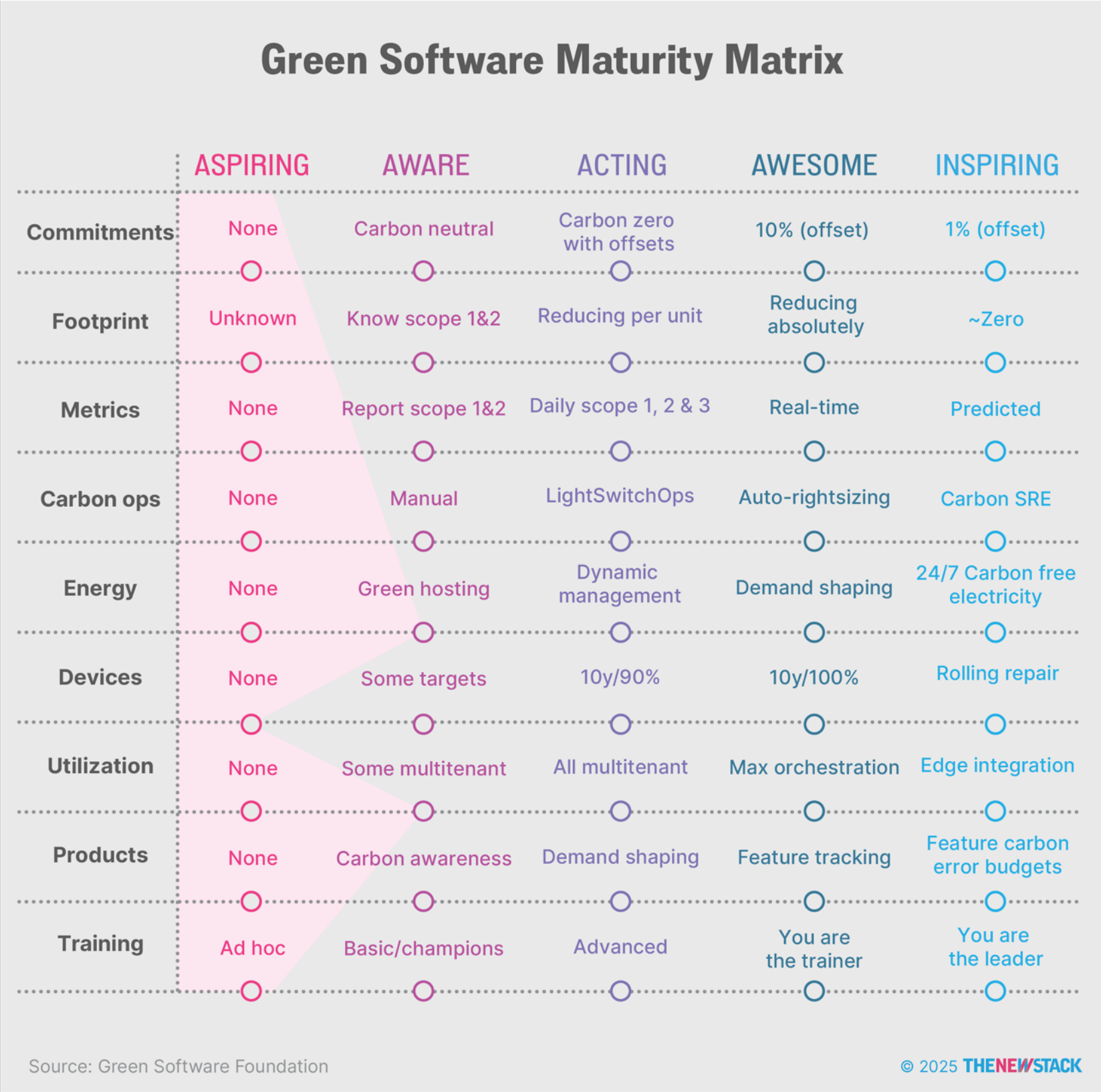


FIG 2.2 The Green Software Maturity Matrix helps organizations rate their “green” initiatives. The light-pink shading indicates that this fictitious respondent is largely in the “aspiring” column, with two dimensions (energy and utilization) in the “aware” category.

If you’re largely in the “aspiring” column (as most companies — like the one in the example above — probably are), the good news is there are a lot of comparatively easy options that will allow you to make rapid progress; it gets more difficult as you get further toward the right.

Before we look at potential solutions, however, let’s clarify some of the terminology used — specifically the various scopes, and the difference between carbon neutral and carbon zero.

Mapping Carbon Scopes to IT

For our purposes, we can divide carbon emissions into **direct emissions** from burning fossil fuels and **embodied carbon**, which is the carbon used to create, transport, and ultimately dispose of hardware.

[The Greenhouse Gas Protocol](#) — which provides the most widely recognized accounting standards for GHG emissions (formally known as the GHG Protocol Corporate Accounting and Reporting Standard, or “Corporate Standard” for short) — categorizes GHG emissions into three scopes.

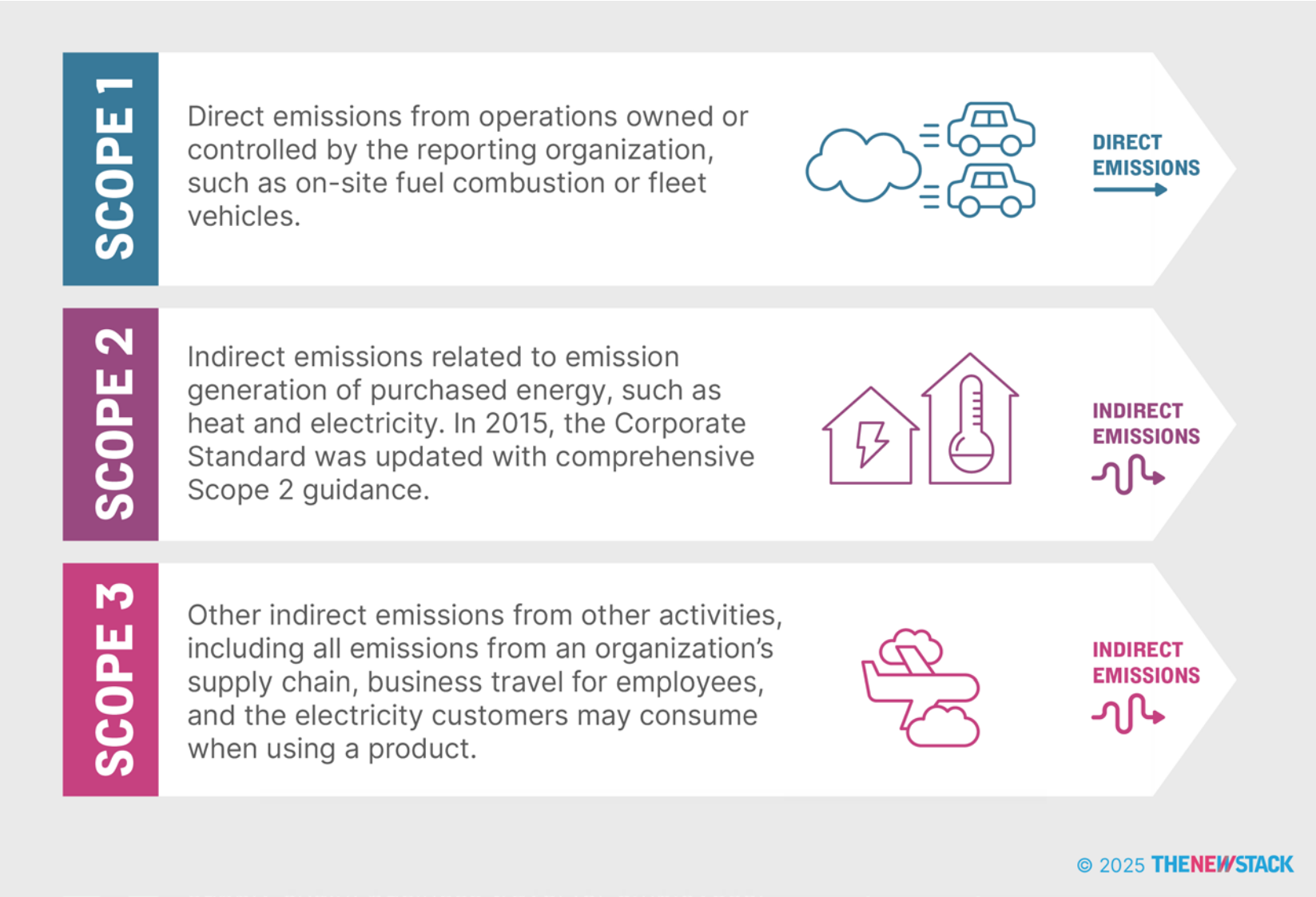


FIG 2.3 The Greenhouse Gas Protocol divides emissions into three scopes based on their source.

Scope 3, sometimes referred to as **value chain emissions**, is the most significant source of emissions and also the most complex to calculate for many organizations.

The protocol doesn't easily map to software. But in the most basic terms, for software running in your own data centers, energy use falls into Scope 2 and embodied carbon into Scope 3.

For public clouds, both your application's energy use and embodied carbon fall into Scope 3. If you are running hybrid, then some energy is in Scope 2 and everything else is Scope 3; frontend user interface consumption all falls under Scope 3 since the code is running on end-user devices elsewhere.

GHG SCOPE	2	3
Private Cloud	Energy	Embodied
Public Cloud	-	Energy + Embodied
Hybrid Cloud	Some Energy	Some Energy + Embodied
Frontend	-	Energy + Embodied

Source: Green Software Foundation.

© 2025 THE NEW STACK

FIG 2.4 Energy use falls into different GHG scopes depending on where it's running and what it's used for.

Carbon Neutral vs. Carbon Zero

Two other terms that sometimes cause confusion are “carbon neutral” and “net zero”:

- To be **carbon neutral**, an organization must measure its Scope 1 and 2 emissions, then match the total with emissions offsets through carbon reduction projects. There is no requirement to cover Scope 3 to be carbon neutral, but most businesses do also offset business travel, which falls under Scope 3.
- To be **net zero**, an organization must be reducing emissions according to the latest climate science, and balancing remaining residual emissions through carbon removals rather than offsets.

How To Achieve an “Inspiring” Rating

To get to the inspiring column on the far right of the maturity matrix, organizations need to achieve two major goals:

1. **24/7 carbon-free electricity in data centers:** This means all electricity use in data centers is time-matched with green grid power. This is Google’s stated goal for 2030, and it is extremely ambitious. This means that for every hour in every location where Google operates, it aims to power all its services through renewable energy.
2. **10-year life expectancy on all end-user devices:** Make hardware last longer and use it efficiently to reduce embodied carbon.

The implication of this is that for data centers the emphasis is more on the direct, rather than embodied, carbon; the reverse is true for end-user devices. This is because servers have become more efficient over time, and the power they consume is less significant relative to their embodied carbon. The upshot is that you can't easily boil it down to a simple rule — you need to run the calculations on your particular servers.

It is much more straightforward for a laptop or mobile phone: Its embodied carbon [dwarfs the carbon](#) used to charge it during its lifetime. In the case of the iPhone 14, for example, 79% of carbon emissions came from production, with a future 2% for transport. Only 18% of emissions associated with the device result from using it.

Few of us use a hardware device for 10 years today. If you are involved in procurement of laptops, mobile phones, and the like for your organization, look at buying machines that have the longest lifespan — both in terms of how well they are constructed and for how long they receive software updates — and put pressure on your vendors to extend that lifetime. If you develop software for end-user devices, try to ensure that it continues to work for the entirety of their lifespan.



CHAPTER 03

The Developer's Role in Sustainability

Coding Efficiency vs. Operational Efficiency

You might have noticed that “coding in more efficient languages” doesn’t feature on the [Green Software Maturity Matrix](#) described in Chapter 2. As programmers, we tend to think of coding efficiency first. But the truth is that this and language choice aren’t high priorities for most of us where sustainability is concerned.

That isn’t to imply that language or framework choice can’t make a significant difference. [Python is about 75 times slower than C](#), although [work is being done to improve this](#).

Python currently uses about 75 more CPU operations than C does to perform the same work. A 75-fold improvement is obviously significant, but in practice, developers tend to favor languages like Java, C#, Go, and Python simply because they are more productive. Except in cases like algorithmic trading systems, where execution speed is the most important thing, businesses generally favor higher-level languages, because what matters to them is time to value (TTV) — essentially, how quickly you can get from having an idea to using it in production.

What's more, there are so many optimizations you can make at the operational level, particularly when running on a public cloud, that optimizing code for sustainability usually isn't worthwhile. If you are running some or all of your workloads

on a public cloud, you are likely better off selecting green regions, concentrating on improving your operations, and taking advantage of the tools your cloud provider offers.

Conversely, if you don't have good operations, the effects of [energy proportionality](#) (explained in Chapter 1) mean that any gains you make by rewriting your apps in a low-level language will be minimal. This explains why, in their book "[Building Green Software](#)," Anne Currie, Sarah Hsu, and Sara Bergman advocate taking an operations-first approach to sustainability.

That doesn't mean that developers don't have a role — far from it. It is simply that your starting point should be focused on operational efficiency rather than at the coding level.

Ways To Improve Operational Efficiency

The following are some of the strategies you can use to increase the efficiency of your ops.

Manual Rightsizing

One area to focus your attention on is rightsizing. This typically means downsizing machines that are larger than necessary and closing down servers that aren't needed. Doing so could potentially cut your emissions in half.

The reason for this is partly down to overprovisioning, but also because of zombie servers — applications that were useful at some point, but no longer are — that haven't been decommissioned. “Building Green Software” includes an example from Martin Lippert, who leads the Spring

Tools projects at VMware by Broadcom:

“In 2019, VMware consolidated a data center in Singapore. The team wanted to move the entire data center and therefore investigated what exactly needed a migration. The result was somewhat shocking: 66% of all the host machines were zombies.”

This is bad for the environment, of course, and it also affects your bottom line since you are paying to run cloud servers or services that you're not using. For workloads in private data centers, zombies also tend to be bad for your security posture because you may be less vigilant about monitoring, patching, and maintaining those machines.

In “Building Green Software,” Ed Harrison, former head of security at Metaswitch Networks (now part of Microsoft), said: “Some of the biggest cybersecurity incidents in recent times have stemmed from systems which no one knew about and should never have been turned on.”

“Try to see if you can halve your cloud bill or the number of servers you are running on-premises; you might be surprised at what you can achieve with some focus.

It can be difficult to spot a zombie server since they look the same as any other machine. One option is to perform a scream test (turning the service off and seeing if anyone screams). While it's possible to automate scream tests (and some organizations do), most people are understandably wary of performing one.

An alternative is to audit the services you are running and see if any of them can be switched off. This sort of work is typically carried out when relocating or moving out of a data center, but it doesn't have to be. Try to see if you can halve your cloud bill or the number of servers you are running on-premises; you might be surprised at what you can achieve with some focus.

[Google Cloud's unattended project recommender](#) identifies projects with low usage over the last 90 days, and might be a helpful starting point. For a more detailed analysis, you can build custom tools using existing data sources in Google Cloud. The [Cloud Asset Inventory API](#) allows users to export all resources in Google Cloud to a BigQuery data set. You can combine these resources with audit logs, which list the operations performed on a resource (e.g.: start, configure, deploy), and monitoring metrics, which show the usage of a given resource.

Google has a few other [cost-optimization](#) capabilities like [Cost Management](#), as do the other major cloud providers. AWS offers [cost optimization](#) with [AWS Cost Explorer](#) and its [pricing calculator](#). And Microsoft has a [range of services](#) to optimize

costs, ranging from [Azure's pricing calculator](#) to [Microsoft's cost management and billing services](#)

Alongside manual rightsizing, take advantage of the autoscaling services built into Kubernetes and offered by all the major cloud providers. They're particularly useful when you have a workload that is usually flat but might surge due to something like an advertising campaign or a Black Friday sale.

There is an issue, though; while organizations are typically very happy to configure their systems to scale up as needed, scaling down is considered a lot riskier, so teams tend not to set this to occur automatically. We need to overcome this reluctance and automatically scale down as a matter of course.

We also need to overcome our collective reluctance to enable low-power mode on our servers. If you've got workloads running on your own data centers, you should enable low-power mode across the board.

LightSwitchOps

A related tactic is switching off the environments you are not using. For example, if you have test or development environments, can they be switched off at night and during weekends? Red Hat's Holly Cummins named this [LightSwitchOps](#), and it appears in the "acting" column of the Green Software Maturity Matrix:

"A while ago I started joking that spinning down clusters was going to be the new 'lights off.'

I predicted that in 2021, every evening when we left the office we'd turn out the lights and then we'd turn off the clusters. It turns out I was incredibly wrong about the 'leaving the office' part. On the bright side, I was more right than I knew about turning clusters off — it's already being done! For example, one team reduced their [cloud costs by 37%](#) with some morning and evening automation."

An issue with LightSwitchOps is that many servers may not behave as expected when rebooted. As former Zynga CIO Dorion Carrol [pointed out](#), this is a sign of poor software hygiene, and it's a really a good idea to take time to address it.

Making Full Use of the Cloud

Your cloud provider offers a range of options and services that are helpful from a sustainability standpoint, but this is one of those areas that goes against the grain. While vendor lock-in is a concern for many organizations, I encourage you to think very carefully about how real an issue it is for your particular circumstances.

By not taking advantage of cloud features due to concerns about lock-in, you make life more difficult for your developers day to day — and make it extremely difficult to meet sustainability goals. Most business applications don't live forever, and replacing them gives you the opportunity to move to a new vendor or cloud provider and start again.

Burstable and Preemptible/Spot Instances

Public cloud providers offer burstable instances, which allow for some automatic management of variance in workload demand. They come with a baseline level of CPU performance, but when the workload's demands exceed it, they can "burst" to a higher level for a limited period. The amount of time the instance can burst is controlled by the number of CPU credits it has.

Burstable instances are less expensive than instances that offer more consistent CPU performance. They are also greener, in the sense that they allow your system to handle occasional spikes in demand without resorting to overprovisioning. However, they do add complexity and, because they will only burst for as long as there are CPU credits available, they are not as safe as overprovisioning.

You might also consider using spot instances (called [preemptible instances](#) on Google Cloud). These are used by public cloud providers to achieve better machine utilization using spare machine capacity. Thanks to energy proportionality, this is better for the environment. They can also be up to 90% less expensive than standard hosting.

They do have some issues you need to be aware of, however. First, preemptible instances are ephemeral, meaning they can disappear at any moment. As a consequence, any software you run on preemptible instances, or software that talks to services or applications running on them, must be able to handle this possibility. They are also not guaranteed: If you put your job in a preemptible instance, it will likely complete eventually, but it may not.

Given that, you might wonder why you'd use them at all. In "[The Cloud Native Attitude](#)," Anne Currie and Jamie Dobson looked at one company, Skyscanner, that used spot instances extensively. It did so not because it was better environmentally or economically, but because it forced developers to build more resilient systems — [chaos engineering](#) in action. It was coincidental that it saved money and made the company more eco-friendly.

Serverless First

Serverless services such as Google Cloud Run, AWS Lambda, and Azure Functions can also provide sustainability benefits. They are multitenant and encapsulated, have fast instantiation, and are designed to be easy to schedule. Because of their fast

instantiation, they can scale up and down easily, making them an ideal choice for bursty workloads. They are also greener, and the more people that use them, the better they are likely to get.

So a good approach for new systems is to take a “serverless-first” approach: Use serverless for everything you can, and resort to alternatives only when your use case can't be handled that way.

In “[The Value Flywheel Effect](#),” David Anderson, Mark McCann, and Michael O'Reilly describe the impact of serverless:

“A serverless-first mindset enables teams to focus on business outcomes and business impact, not keeping the lights on. It also has added benefits, including keeping your code low, your security high, and your cost liabilities under control. Serverless-first is a game changer that can allow organizations to move rapidly and sustainably. It also removes developer friction.”

Anderson is now [applying the same principle](#) to multiple firms at his new workplace, Globalization Partners (G-P). Other organizations that have successfully used serverless include the [BBC](#), [Cinch](#), and [Honeycomb](#).



CHAPTER 04

Unpacking CPU Performance in the Cloud

A topic that tends to get less attention than it deserves regarding sustainability is choice of CPU. I think this may be because it is, to a large extent, abstracted away in the public cloud. But CPUs need electricity to run, and they have embodied carbon.

The good news is that CPUs are getting better at energy proportionality. These improvements are part of what cloud vendors have been able to exploit to increase their profitability. But you can take advantage of them too.

Virtual machines (VMs) running AMD processors in the public cloud are also cheaper and more efficient than Intel ones. “It is more cost effective to run on AMD, period,” Allen Leibovitch, senior manager of cloud product marketing at AMD, told me. “We can prove this, and we have done so over and over again. Likewise, if the developer is selecting an AMD-based VM, this is also better for the environment. And from a developer point of view, things are going to run faster and better on AMD.”

Performance Comparisons

Leibovitch would say that, of course, but this is a fair statement because AMD offers more cores per chip than Intel, which means more processing per server, equating to fewer servers for the same workload. That core density advantage means only 11 AMD EPYC-powered servers are needed to run 2,000 VMs vs. 17 Intel-powered servers. This can result in using [35% fewer servers and 29% less power](#) annually, which helps Google Cloud meet its environmental impact commitments.

Like Google, AMD also has made strong public environmental commitments, including announcing a goal [to achieve a 30-fold increase in energy efficiency](#) over its 2020 baseline by 2025. This would equate to a 97% reduction in energy use per computation in five years.

[ARM processors](#) have the potential to be more efficient still, and we are seeing another surge in the use of ARM in data centers — but AMD has the advantage of x86 compatibility. Leibovitch contends that AMD will generally outperform ARM in the cloud, and he further pointed out that in most cases x86 migration requires no retooling or refactoring, with multiple customer and partner programs available to help facilitate the transition.

From a cost perspective, AMD provides [up to 13% cost savings](#) over other comparable Google Cloud VMs. The AMD-powered C3D VM brings 29% cloud OPEX savings over previous-generation Intel-based N2 VMs. AMD also claims that [C3D VMs](#) [boost](#) average performance by 44% and reduce cloud operating expenses (OPEX) by 25% (compared to previous-

generation AMD-powered N2D VMs). Of course, both AMD and Intel are releasing newer chips frequently, so you need to check these numbers for the latest versions. But by default, picking AMD over Intel is likely to produce cost savings regardless of which cloud provider you are using.

AMD also has a strong security story in its [Infinity Guard](#) security system, which runs entirely in hardware. “It is based on a hardware offload engine that is doing encryption in use,” Leibovitch said. “This means that you are encrypting the data as it is being processed between the CPU and the memory, so even Google can’t see your data.”

For a small fee, you can enable it by literally clicking a box in the cloud console, and the performance hit is minimal. Google offers it on both of these VMs, and uses it for [Confidential Google Kubernetes Engine \(GKE\) nodes](#), as well as [Confidential Space](#).

“To gain the most efficiency and sustainability from your cloud instance, it is important to match your workload to an appropriate VM instance.

Matching Workloads to VMs

To gain the most efficiency and sustainability from your cloud instance, it is important to match your workload to an appropriate VM instance. This table offers examples of recommended workloads for AMD VM instances operating on Google Cloud:

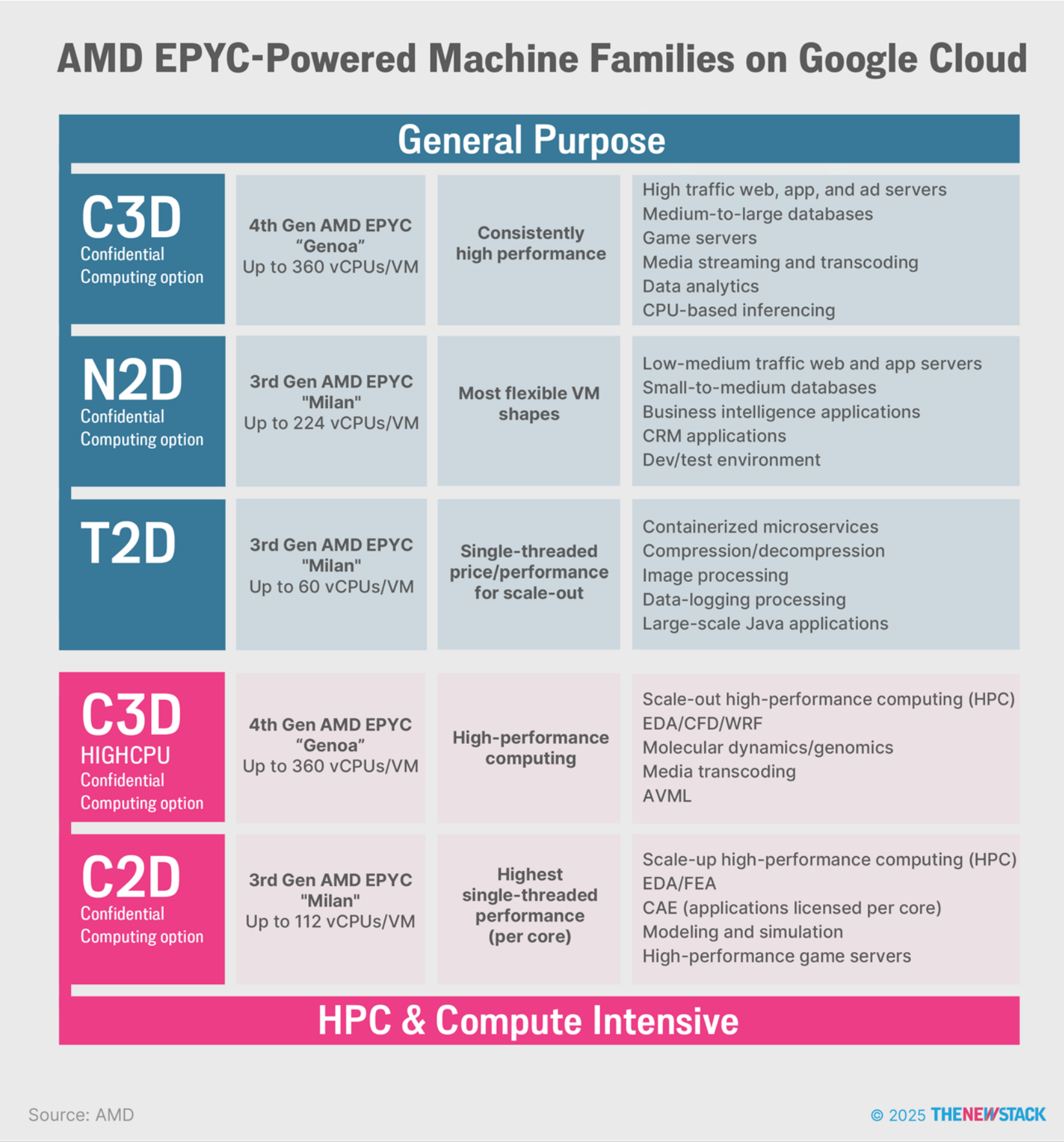


FIG 4.1 Recommended workloads for AMD VM instances operating on Google Cloud Platform. EDA=electronic design automation; CFD=computational fluid dynamics; WRF=weather research and forecasting; FEA=finite element analysis; CAE=computer-aided engineering. (Source: AMD)

For more compute-intensive workloads, Google also offers a C2D VM based on the 3rd Gen AMD EPYC with up to 112 vCPUs per VM, and the C3D highcpu version can handle scale-out high-

performance computing (HPC) tasks. In addition, AI inference, which is typically handled at the edge in the public cloud, can be done on CPUs.



CHAPTER 05

Measuring Carbon Emissions

The approaches we've looked at so far are pragmatic and relatively easy to employ. They don't require detailed measurement, which is good because accurate real-time information on carbon emissions remains hard to come by. This means that we often need to rely more on proxies to calculate our environmental impact.

CPU and Hardware Utilization

One of the best proxies for energy is CPU utilization, which is easy to instrument. Hardware utilization also works well, although it isn't as straightforward. Producing new hardware has a considerable embodied carbon cost. In addition, due to energy proportionality, running more servers at lower utilization consumes more carbon than a smaller number of servers at higher utilization. Newer machines are also more efficient, and there are a number of system effects you need to consider.

Cloud Costs

In [“How To Avoid a Climate Disaster,”](#) Bill Gates calculates the “green premiums” — the comparative cost of doing something in a sustainable way — for various industries. I'll let Gates introduce it:

“There isn't one single Green Premium. There are many: some for electricity, others for various fuels, others for cement, and so on. The idea of the Green Premium depends on what you're replacing and what you're replacing it with. The cost of, say, zero-carbon jet fuel isn't the same as the cost of solar-generated electricity. I'll give you an example of how Green Premiums work in practice.

“The average retail price for a gallon of jet fuel in the United States over the past few years is \$2.22. Advanced biofuels for jets, to the extent they're available, cost on average \$5.35 per gallon. The Green Premium for zero-carbon fuel, then, is the difference between these two prices, which is \$3.13. That's a premium of more than 140%.”

In rare cases, a green premium can be negative — that is, going green can be cheaper than sticking with fossil fuels. Gates doesn't provide a number for the IT industry, but I think it is one of those rare cases where the green premium is a negative number. The reason for this is that, although there is work for us as an industry to do to improve our green credentials, the greener systems we use are generally cheaper, so the "premium" is not significant.

Given this, cost is a useful proxy for carbon emissions — up to a point. Limitations include the fact that bills from cloud providers do not factor into your workloads' carbon awareness. And electricity cost is not necessarily linked to the greenness of its production. In general, though, lowering your costs will lower your carbon emissions. It will also likely gain the support of your CFO,

which can be helpful if you need C-suite buy-in for a sustainability initiative.

“By disclosing both Scope 2 market-based to location-based emissions, we allow customers to make better decisions and drive and track real behavioral change based on different dimensions — Cynthia Wu, Google

Assessment Tools

Beyond proxies, [AWS](#), [Azure](#), and [Google Cloud](#) all have strong sustainability goals, and provide tools to assess your carbon footprint on their platforms with varying degrees of capability. [Cloud Carbon Footprint](#) is an open source alternative sponsored by Thoughtworks that can also measure, monitor, and reduce your carbon emissions on public clouds.

Google's Carbon Footprint Tool

Google's Carbon Footprint is a report available via the Google Cloud console. It offers a high-level dashboard-style overview of a customer's carbon emissions. Google also provides a series of video tutorials in the tool, plus an [extremely detailed breakdown of its methodology](#), as well as an

[academic paper](#) that goes into further details of the work behind the methodology.

The report covers the two major ways of accounting for carbon:

- **Location-based reporting** only accounts for the grid — that is, it shows how clean or dirty the grid is where the data center resides.
- **Market-based reporting** accounts for the provider's additional renewable energy procurement. The figure for market-based will typically be a lower number than location-based.

At the time of writing, most other cloud providers offer only market-based, not location-based, reporting.

“By disclosing both Scope 2 market-based to location-based emissions, we allow customers to make better decisions and drive and track real behavioral change based on different dimensions,” Cynthia Wu, senior product manager at Google Cloud Carbon Footprint and Sustainability, told me in an interview.

The report shows both monthly and annual carbon emissions, broken down by region (for example: asia-east1, us-east-1), product (such as Kubernetes Engine, Compute Engine, or Gemini-API), and customer-defined project. “This level of granularity helps our customers dig deep to find where they should focus their optimization efforts,” Wu said.

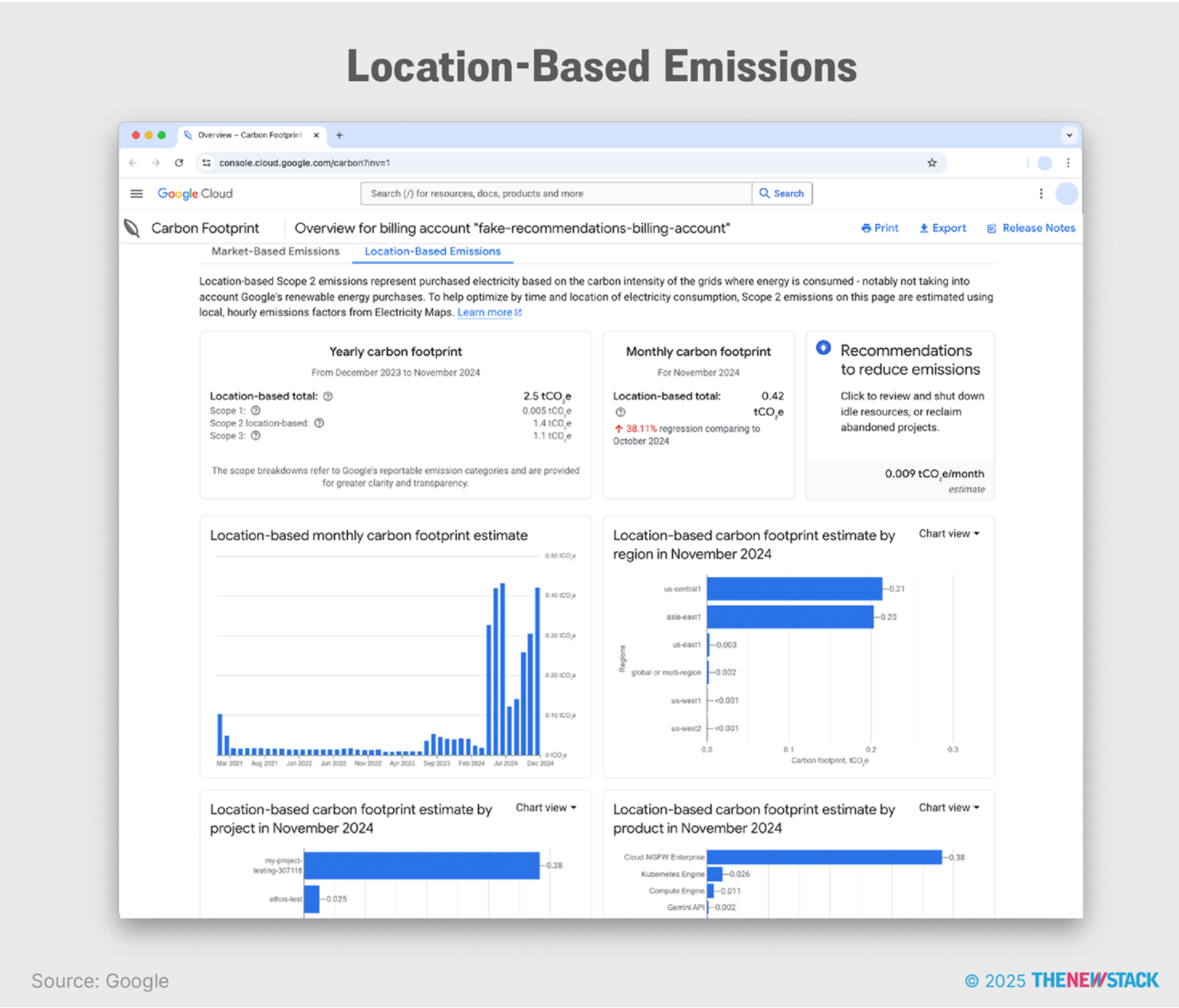


FIG 5.1 The Location-Based Emissions tab on Google’s Carbon Footprint report shows the impact of the local grid on emissions.

The yearly carbon footprint report in the image above categorizes emissions according to the [Greenhouse Gas Protocol scopes](#). In this case:

- **Scope 1** will typically be things like Google’s backup diesel generators, natural gas for heating, and fuels used in fleet vehicles. It will be a low figure.
- **Scope 2** is the electricity used to run all your workloads, as well as ancillary electricity services such as cooling and lighting.
- **Scope 3** is where Google accounts for the embodied carbon from the construction of data centers and the manufacturing of the equipment in them, as well as employee travel for data center staff.

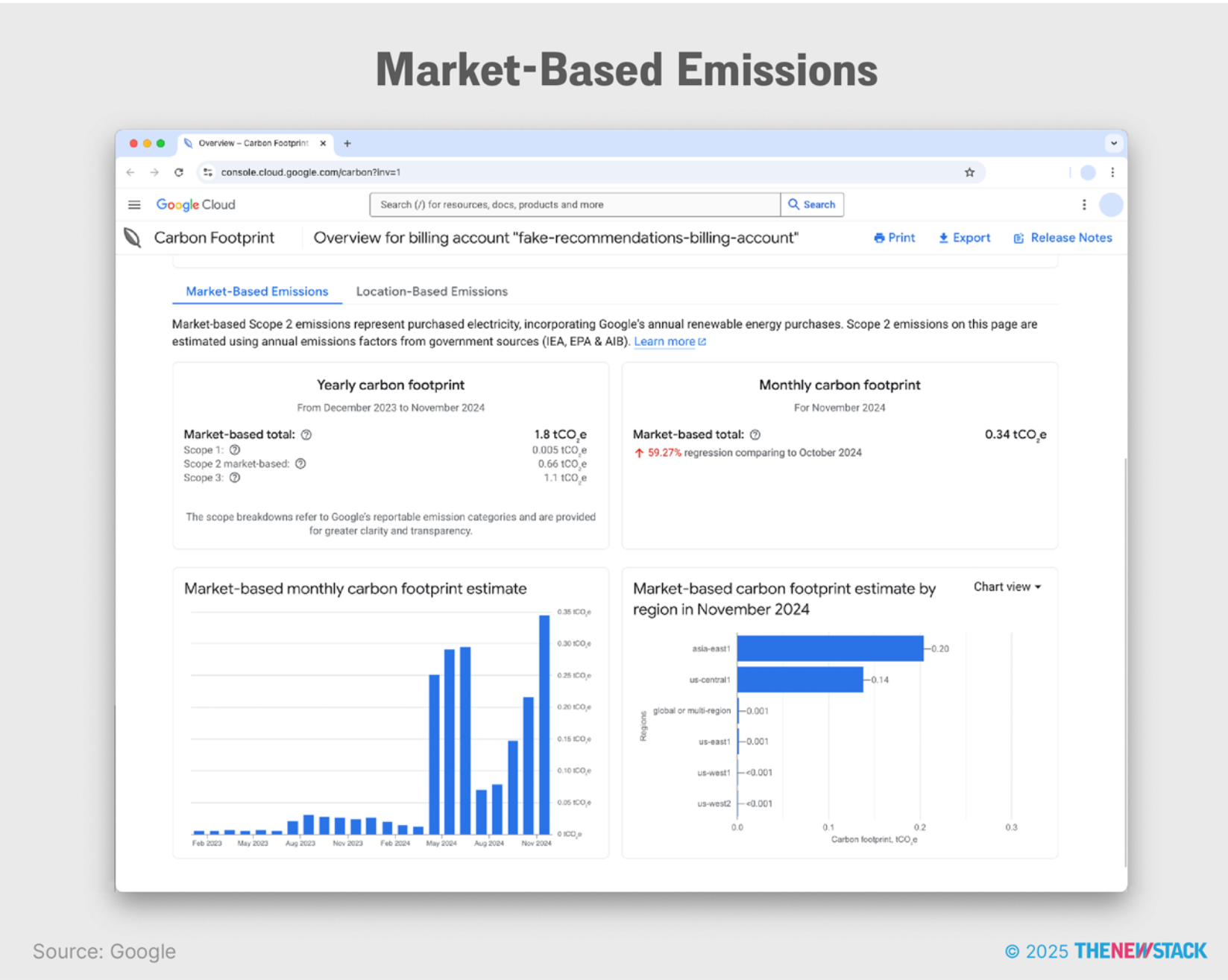


FIG 5.2 The Market-Based Emissions tab provides data on the provider’s renewable energy procurement.

Although originally targeted at sustainability officers, Google has noticed an expansion in audience for the Carbon Footprint tool over time. “Increasingly, we’re seeing GreenOps emerging within FinOps practices, meaning that those customers who

want to optimize for cost also now want to optimize for carbon,” Wu said. Reflecting this, Google’s Carbon Footprint team has started collaborating with the Google FinOps team to add carbon insights into the FinOps hub:

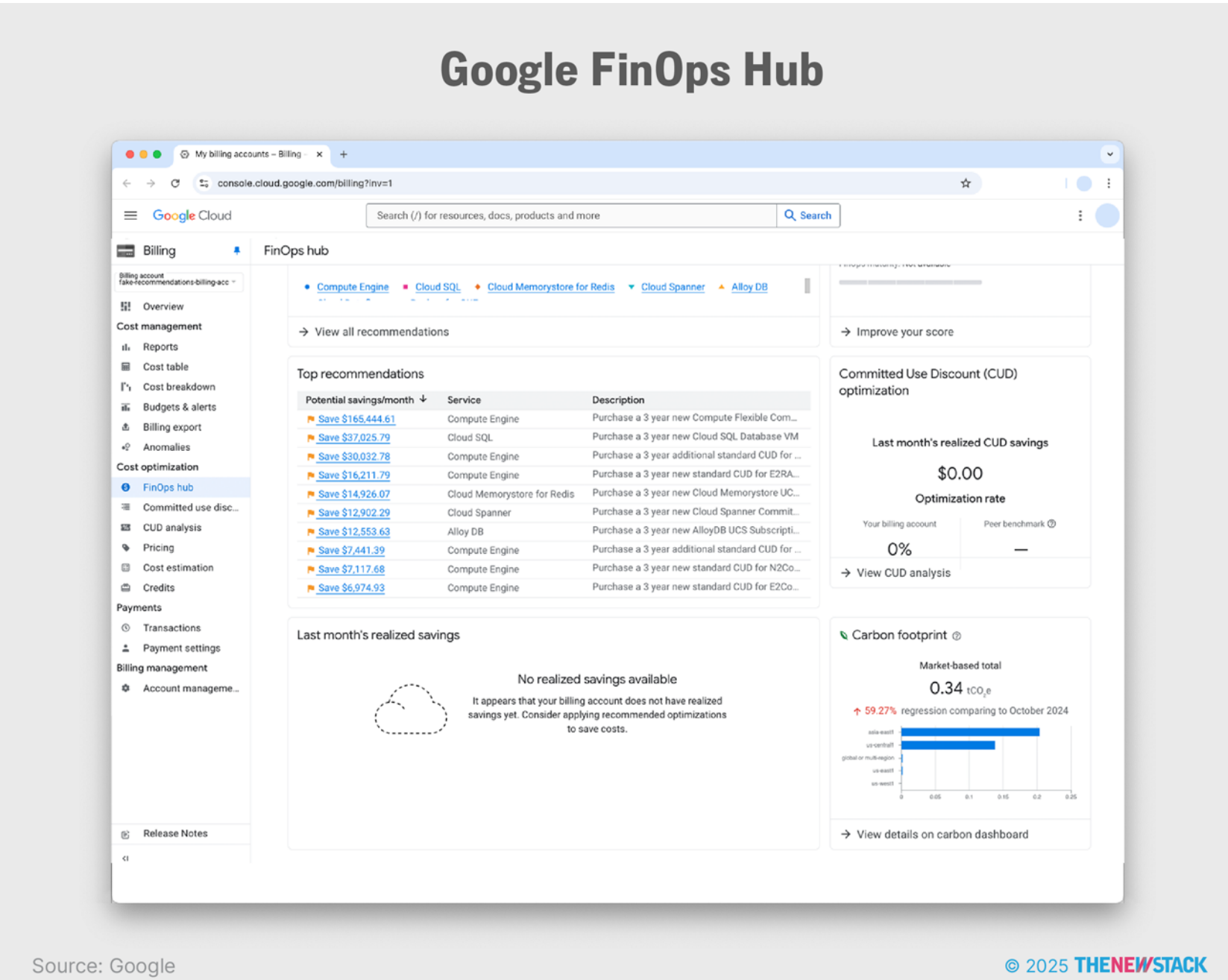


FIG 5.3 Caption: Google’s FinOps hub now includes carbon footprint data.

Alongside the dashboard, Google also allows customers to export their data to BigQuery, offering greater granularity.

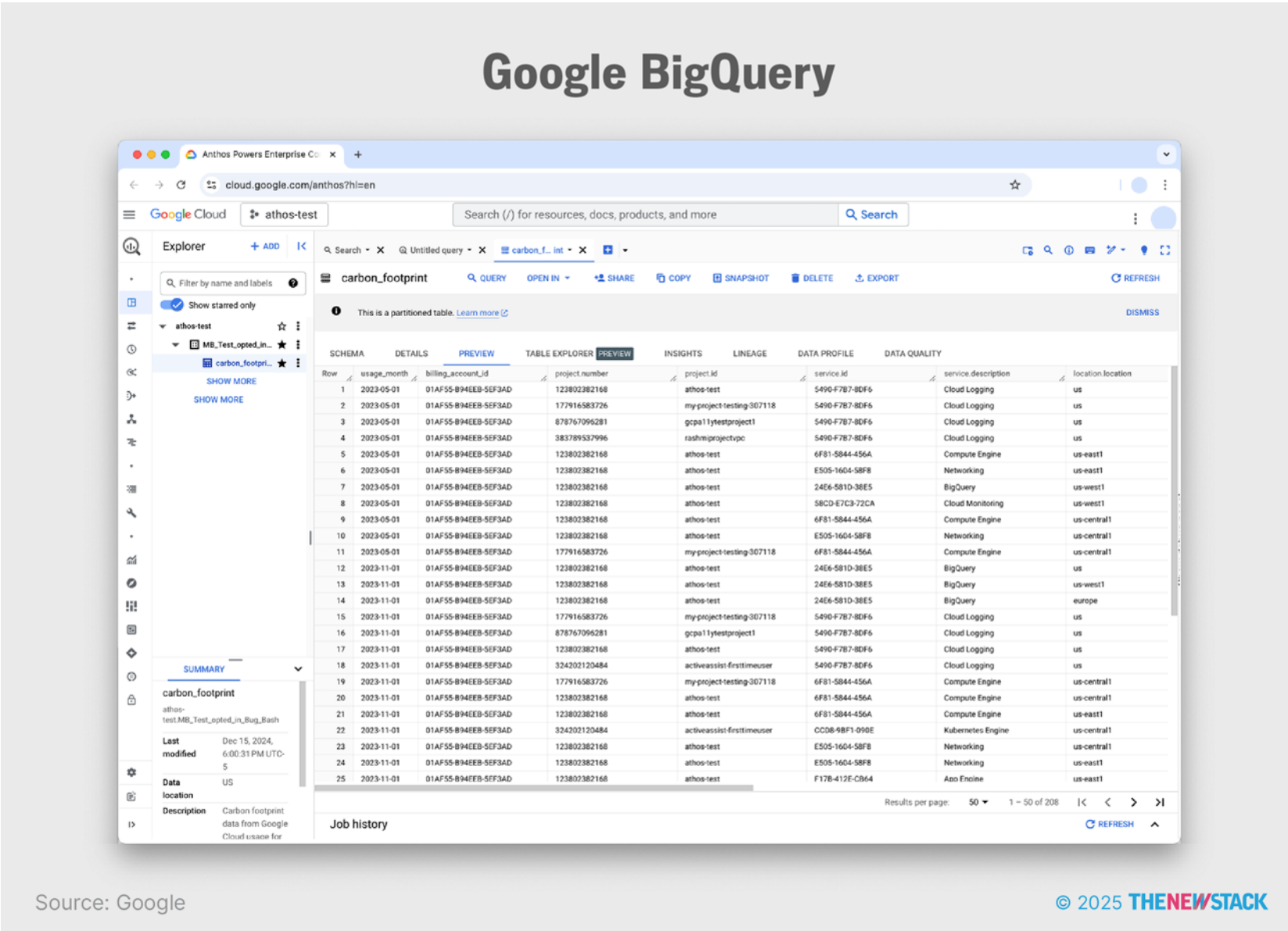


FIG 5.4 Data can be exported from Google’s FinOps hub to BigQuery for more detailed analysis.

“From BigQuery, you can export the data to [Google] Sheets, Looker, or your analytics tool of choice to really analyze the data,” Wu said. “You can also merge

it with your billing data so you can look at carbon emissions per dollar spent, which can give a carbon cost ratio.”

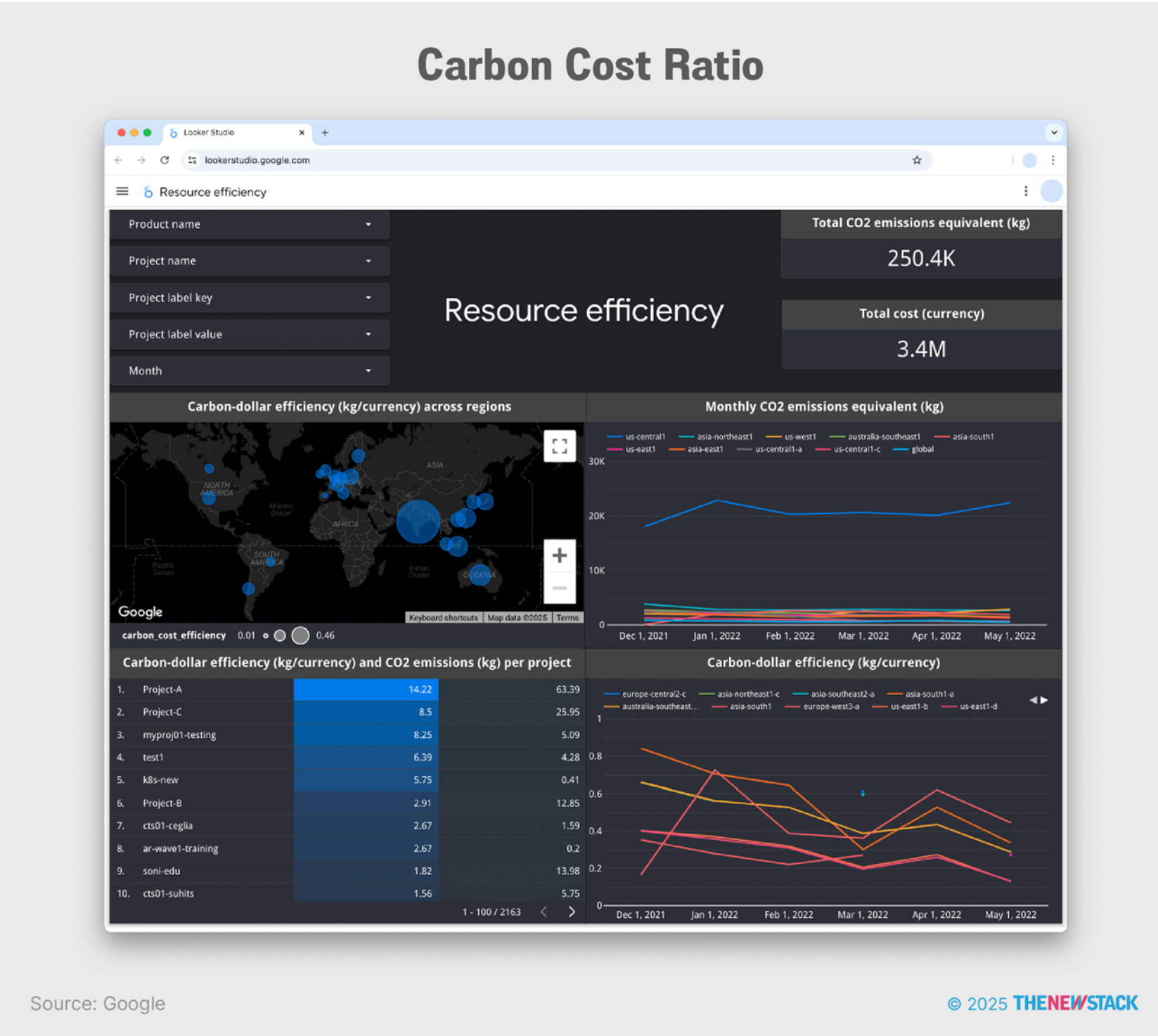


FIG 5.5 You can use your carbon emissions data and cloud billing data to calculate your carbon cost ratio.

Google continues to invest in the product and respond to customer demand, with many customers asking for more granularity, Wu said. “As customers become more educated on this topic they’re digging deeper and trying to optimize everywhere they can in their architecture,” Wu said. “We already help them on a regional basis, but customers are asking for help regarding the carbon intensity between, say, an N2 and C3 VM. This is something we’re thinking about how to enable.”

Green Software Foundation Carbon Intensity Formula

The Green Software Foundation has a [formula for calculating carbon intensity](#). The method is akin to a product carbon footprint or life cycle assessment (LCA) approach, and is not intended for corporate-level accounting or reporting.

At its simplest, it shows your Software Carbon Intensity (SCI), or how much carbon is created for each functional unit of an application. This is the full formula:

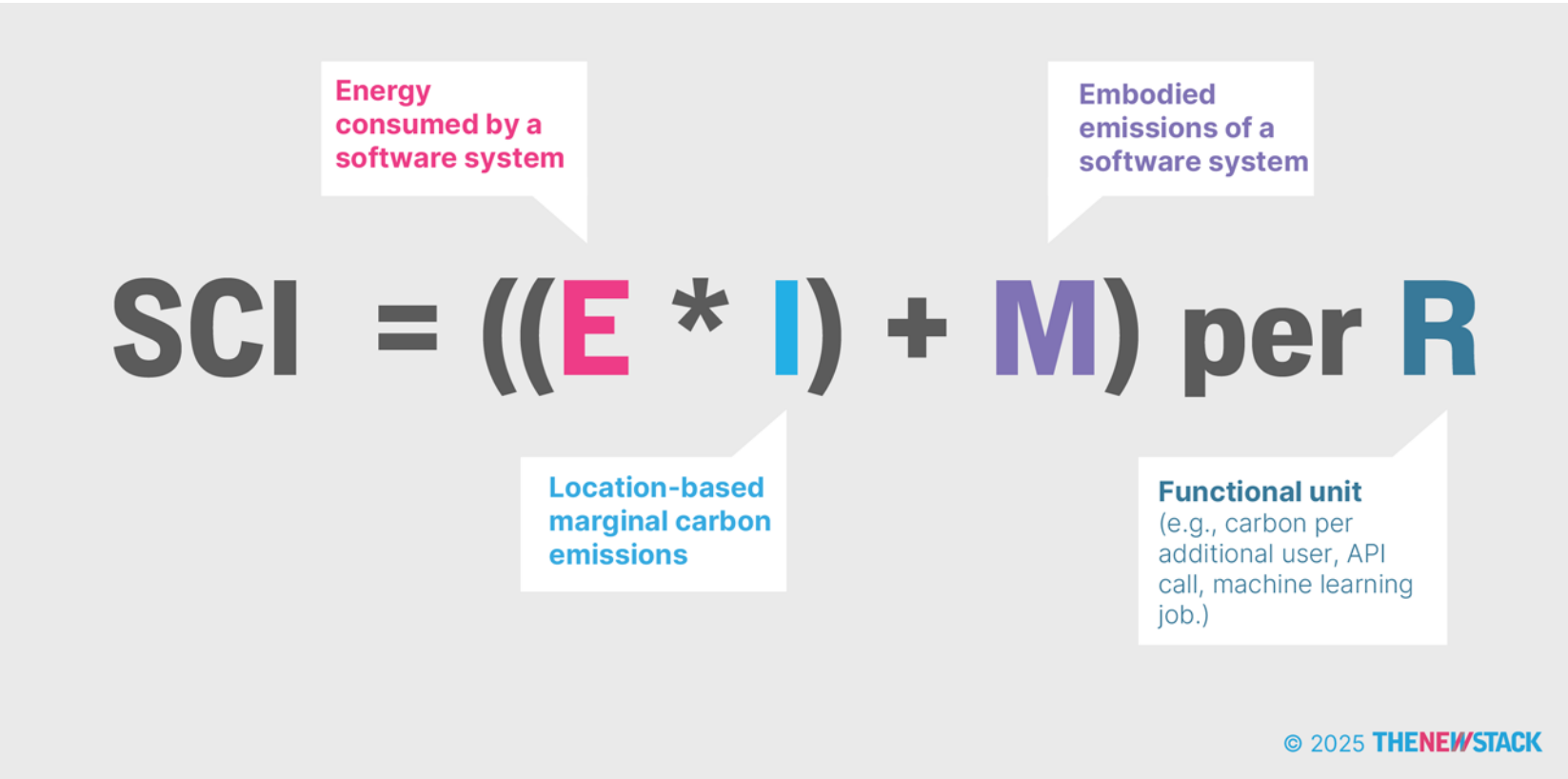


FIG 5.6 The Software Carbon Intensity score calculates how much carbon is created for each part of an application.

In the formula **SCI = ((E * I) + M) per R**:

- E: Energy consumed by a software system
- I: Location-based marginal carbon emissions
- M: Embodied emissions of a software system
- R: Functional unit (e.g.: carbon per additional user, API call, machine learning job)

This formula shows the importance of using less electricity and fewer physical

resources to perform a function. It also highlights the fact that location matters. Moving to carbon-free electricity sources is a slow process and, inevitably, different locations will get there at different stages.

Also consider that carbon intensity changes over time. Typically, electricity is produced so that supply always meets demand. Electricity storage is difficult, and since it has been largely unnecessary, there is currently little storage or other buffering in electricity grid systems.

This is a problem for renewables because their energy sources are less predictable than fossil fuels. When it's cloudy or the wind isn't blowing, carbon intensity increases because more of the electricity in your mix has to come from fossil fuels. When demand spikes, it is relatively easy to cope by turning to a fossil fuel source: You simply burn more coal and produce more electricity. Renewables lack this flexibility. So in times of peak demand, you typically have to fall back on fossil fuels for part of the electricity mix. Or you need to have enough wind turbines and solar panels to deal with peak demand, then throw away much of that power because most of the time you don't need it.

This dispatchability problem is one of the reasons why nuclear power, another carbon-free source, could be a

necessary, if controversial, part of the energy mix for the foreseeable future. Nuclear energy is traditionally considered a non-dispatchable generation technology; however, certain modern nuclear reactors can [potentially act as a dispatchable power source](#). (The other reason has to do with energy density; essentially, we need more land mass to meet all our electricity needs from renewables.)



CHAPTER 06

Taking Steps To Reduce Emissions

Eventually, we'll have 100% carbon-free electricity everywhere. But in the meantime, if we want to use fewer fossil fuels, we'll need to take proactive steps to do so.

For a really quick, easy win, particularly if you are running workloads on a public cloud, you can choose the region with the greenest electricity mix. “Some of our customers are moving workloads — typically batch jobs or training models where they don’t have latency requirements — to where the grid is cleaner or Google is procuring a lot more green energy,” Wu said. “They are balancing this against their performance needs and cost optimization.”

[Electricity Maps](#) has provided a tool that outlines this information. It has partnered with Google, AWS, and Salesforce to provide real-time, global coverage of the carbon intensity of electricity. If you are thinking about building a new on-premises data center,

it is probably the first thing you should look at. Google Cloud also provides the [necessary data](#) to support this decision-making process.

We can go further, though.

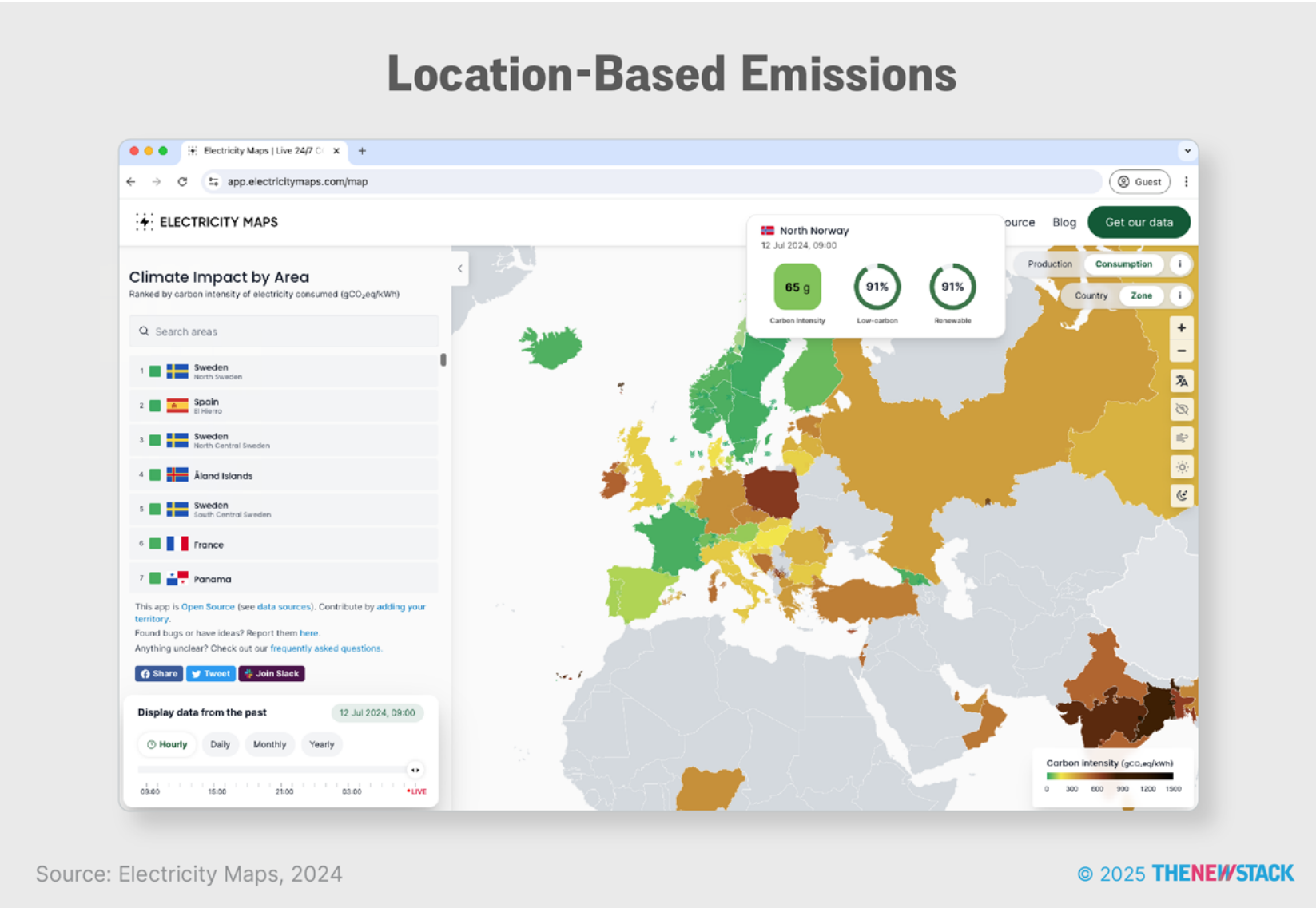


FIG 6.1 Electricity Maps shows real-time, global coverage of the carbon intensity of electricity in different regions.

Demand Shifting

There are APIs available to help you look at carbon intensity, including from [Electricity Maps](#), [WattTime](#), the GSF's [Carbon-Aware SDK](#), and, in the UK, the [Carbon Intensity API](#). Armed with these, you can apply a more advanced technique called “demand shifting” — responding to changes in carbon intensity by increasing or decreasing your demand.

Demand shifting can be done based either on time or location. Time is probably easier and is likely more familiar. If you've ever had batch jobs that run overnight, you are thinking along the right lines. Wu said, “4 p.m. to 9 p.m. is typically when the grid is dirtiest because everyone goes home, starts cooking, and turns on their lights, and so the grid needs a lot of capacity. Whereas noon in California has a lot of solar power on the grid and not enough demand to use it all.”

Batching is one approach we can use, but since microservices became fashionable, we have all built distributed systems whether we needed to or not. Fortunately, several other tools that were developed to engender more resilience in distributed computing can equally be applied in a carbon-aware way. For example, with any event-driven architecture, you can push workloads into streams to be processed — and dealt with when needed — by different components. The benefit of this approach is that it's possible to offload these components to other regions where they can be batched up or run at later times.

The approach is particularly applicable when a use case isn't latency sensitive. AI training is a good example. If you are training a large language model (LLM), you could choose to do so only when the electricity is greenest.

Graceful Degradation

A related approach, called “graceful degradation,” is used when a microservice doesn’t respond quickly enough, so instead of the correct response, it offers a slightly stale result retrieved from a cache, if there’s a suitable one available, or no response.

Netflix used this a lot, particularly in its early days when its services were less resilient, before it discovered the magic of [FIT/chaos engineering](#). If, for example, Netflix can’t reach the movie recommendation service to retrieve up-to-date recommendations, it will instead fetch some slightly stale results from a cache and show you that instead. You probably won’t notice, and even if you do, it will just be slightly puzzling — “Why on earth is it recommending that again?”

Other parts of the Netflix homepage that are less essential may not display at all if the service can’t be reached. As a software engineer, you’ll probably notice these things, but many of the changes are so small that they’re almost imperceptible to the vast majority of users.

Of course, there is no reason why you couldn’t apply graceful degradation in a carbon-aware way — for example, by performing a cache discard and refreshing the movie recommendations list only when the electricity you are using is (or is projected to be) greenest.

The pushback you receive might be based on commercial pressures — “We need to get this model built fast so we’re not behind our competitor” — but taking longer to do something the right way may pay dividends. If you say so publicly — “We’re taking a little longer to develop our GenAI strategy as we want to do it in a sustainable way” — some of your customers might be more likely to wait for your product to ship. And by not rushing, you can avoid some of the pitfalls that happen when you ship a product before it is ready.

Location-Based Demand Shifting

Demand shifting can also be location-based. For example, to reduce carbon intensity, you could move to different hemispheres seasonally to maximize exposure to sunlight and solar power.

It's analogous to having call centers in different regions so that you can offer 24/7 support without needing people to work unsocial hours.

Moving your workloads may be straightforward, but if they depend on large amounts of persistent data, moving that data is harder and slower, and may not be practical. You may also have to deal with data sovereignty laws, which can be another challenging hurdle.

Demand shifting might sound purely theoretical, but there are in fact several real-world examples of using it in practice:

- [Google launched a project to make some cloud workloads carbon-aware](#). It says that it “can now shift moveable compute tasks between different data centers, based on regional

hourly carbon-free energy availability.”

- Microsoft [announced a project](#) to make Windows 11 more sustainable by running Windows updates when the carbon intensity is lower. Another Microsoft product, [Xbox, now performs updates when the electricity from the grid](#) is the cleanest, and its power-saving “Shutdown” mode can cut electricity consumption up to 20-fold.
- Apple has [Clean Energy Charging](#) for U.S. iPhone customers that uses a forecast of the carbon emissions in your local energy grid in order to charge your iPhone during times of cleaner energy production.

Green software architecture requires savvy and should not be underestimated. Keep in mind that cloud providers monitor where workloads are and may wish to build a new data center if they see a spike in demand. It would run counter to your sustainability goals to encourage anyone to pour more concrete and build new servers, given all the greenhouse gas emissions that will be generated, especially if you are going to be in the data center for a short time. Network traffic has a carbon cost, and locations have performance implications.

Another trade-off is, wherever possible, to present information from a location as close to the user as possible, and minimize requests traversing the globe. Content delivery networks (CDNs) are one way to achieve this. Caching content within a CDN will require fewer

requests to be processed and transported around your data center, with the added benefit of reducing latency for the user.

Demand Shaping

Demand shaping is similar to demand shifting, but instead of moving demand to a different region or time, we shape customer demand in line with the carbon intensity of the existing electricity supply. As such, it may be more applicable in cases where workloads are more latency sensitive.

[ClimateAction.tech](#) is using this approach, also known as “demand-responsive” design, with its “[Branch](#)” magazine. Sustainable digital product designer Tom Jarrett [described in a blog post](#) how the organization uses

data from a grid intensity API along with the user's location to "adapt to and reflect the physical infrastructure of the internet and the energy behind it."

"Branch" is the only real-world example I've found, but there are plenty of other places where we do similar things for other reasons.

Videoconferencing software shapes demand by reducing video quality and prioritizing audio. This was particularly obvious during Covid lockdowns, when use of tools like Zoom increased by a staggering amount. We sometimes looked a bit pixelated during our conference calls, but it was mostly fine.

Video streaming services like Netflix use adaptive bitrate streaming to achieve a similar result. The idea is that multiple versions of video files are

created and encoded ahead of time to fit a variety of network conditions. Then, based on factors like bandwidth and device type, the video player selects the highest-quality file that the device can play, with the smallest amount of buffering possible. Again, during Covid, this happened a lot; our movies were a bit grainier but we could still watch them.

You might also think of examples like progressive enhancement in web design, where the web experience has more bells and whistles that affect the resources and bandwidth of the end-user's device, or TCP/IP, where the transfer speed increases in response to how much data can be broadcast over the wire.

Demand shaping can be largely invisible, as it is in these examples, but

you could also give control of it to users of your application or visitors to your site. If you use a washing machine or dishwasher, it probably has an eco cycle that takes longer but uses less electricity by washing at lower temperatures of 30°C (86°F) or less. Most washing machine eco cycles also use less water by increasing the drum movements to clean your laundry.

On a website, you could include a “low carbon intensity” button or slider that reduces the graphical intensity of the site. You might imagine this would be too niche for most of your users to care about, but you could find that some people use it because they care about their environment while others may genuinely prefer a site with fewer visual distractions.



SUMMARY

AMD and Google Cloud collaborate to provide high-performing, cost-efficient computing solutions tailored to meet a wide variety of workloads, so you can confidently pursue digital innovation in the cloud.

KEY FEATURES

AMD processors for Google Cloud workloads deliver:

- **World-class performance:** AMD-based VMs, built on Google Cloud’s resilient and scalable global infrastructure, are powered by AMD EPYC processors, which hold over 500 world records and deliver exceptional performance to customers.
- **Cost optimization:** AMD’s collaboration with Google Cloud is driving cost savings for customers, with VM users experiencing an impressive reduction in their compute costs.
- **Sustainability:** AMD is dedicated to sustainability, aligning with its mission to empower businesses to achieve more with AMD VMs. By optimizing resources, AMD boosts efficiency and promotes a greener future.



CHAPTER 07

Putting It All Together

If you are looking to start a sustainable computing initiative in your organization, key allies are likely to be:

- Your CFO, because it will save money.
- Your head of security, because it will improve your security posture.
- Your head of recruitment, because there is still a war for talent in IT, and an increasing number of developers care about the broader impact of their work.

For sustainable software, think operations first rather than code.

For workloads in your own data centers, aim to reach more than 50% utilization for servers across your estate.

But regardless of where your workloads run:

- Apply LightSwitchOps, turn off underutilized machines (zombies), and do manual rightsizing.
- Use autoscaling where possible (including scaling down).
- Run more applications on the same hardware (multitenancy).
- Use more efficient CPUs that are matched with your workload requirements.
- Look at where your workloads run from an environmental perspective using Electricity Maps, and apply demand shifting and shaping techniques.

Finally, remember that as a developer,

you have the opportunity to put pressure on your vendors to focus on efficiency and sustainability. Most of the big cloud providers will respond to customer demand, so keep asking them to push toward being carbon neutral without relying on offsets and to provide you with information to make informed choices. If you are using a machine learning model, ask the vendor what its sustainability story looks like; if it won't tell you, consider whether its usefulness outweighs the potential damage it is doing.

With some caution, because being an open source maintainer can be a thankless task, I'd also encourage you to put pressure on the large open source projects you use regarding sustainability. Get involved if you possibly can. Going forward, it is vital that we all do our bit, however small, at home, in our community hubs, and in our workplaces.



CHAPTER 08

Looking Ahead: Trends and Future Considerations

I'd very much like to see all the cloud providers providing similar details about how they arrive at their various carbon calculations. If you are using multiple cloud providers, it's currently extremely difficult to perform a like-for-like comparison or even work out what the differences might be.

Renewable Energy Certificates

The current procedure for reporting greenhouse gas emissions dates back to the 1990s. One aspect is that each time a wind, solar, or hydroelectric facility generates a unit of clean power, its owner can issue an energy attribute certificate, typically known in the U.S. as a renewable energy certificate (REC). These can either come “bundled” into a contract for clean power, or can be bought individually, or “unbundled,” from a generator or market intermediaries.

“Bundled RECs are the ones that Google procures,” Wu told The New Stack. “This means that if we have a data center in Iowa, we procure renewable energy on the same grid where our data centers operate. Many other cloud service providers use unbundled RECs where, hypothetically, you could buy energy from Canada, which has a lot of hydropower and is therefore cheaper than the energy in Iowa. If you apply that to your Iowa data centers, your Scope 2 market-based

emissions would appear to be lower, but it’s not applied to the Iowa grid. This means that it isn’t driving renewable energy generation where the consumption happens.”

She’s not wrong. [Company filings](#) to the independent environmental disclosure system CDP for 2023 show that over half of Amazon’s and Microsoft’s renewable energy came from certificates unbundled from power supply contracts:

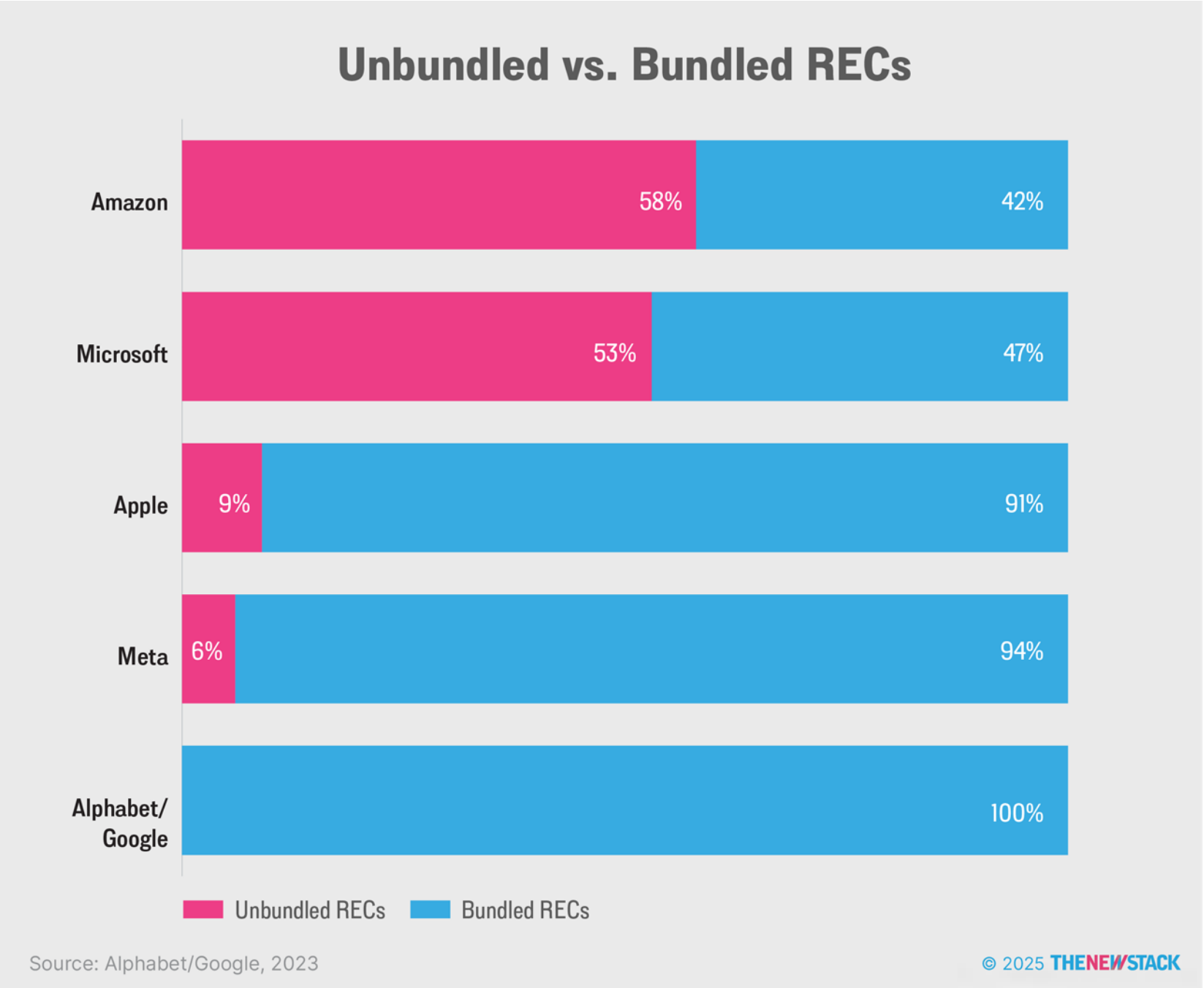


FIG 8.1 Amazon and Microsoft use more unbundled RECs than Apple, Meta, or Alphabet.

There is nothing illegal about buying unbundled RECs, but their use is certainly questionable. Google's advocacy on Scope 2 and renewable energy goes further than location matching. The company also wants to [do matching on an hourly basis](#), rather than annually (as it currently is done), to align with the real-timeliness of energy generation and consumption.

Matching electricity consumption with carbon-free energy sources every hour of every day on each grid Google operates is the goal of Google's [24/7 Carbon Free Energy \(CFE\)](#) initiative. Although it is a far more complex and technically challenging pursuit than annually matching energy use with renewable energy purchases, Google aims to achieve this ambitious standard by 2030.

Google believes its "greatest impact lies not only in advancing CFE for our own operations, but also in accelerating decarbonization of electrical grids around the world." Google also notes that a "shift in focus to 24/7 CFE procurement — matching consumption on an hourly basis from within the same grid — will have a more significant impact on grid decarbonization and the clean energy transition" for the entire world.

The rules matter in part because all the major successes we've seen to date with regard to climate change have resulted from legislation.

In the late 1980s and early 1990s, acid rain linked to sulfur dioxide emissions and the hole in the ozone layer were perceived as the major threats, rather than greenhouse gases.

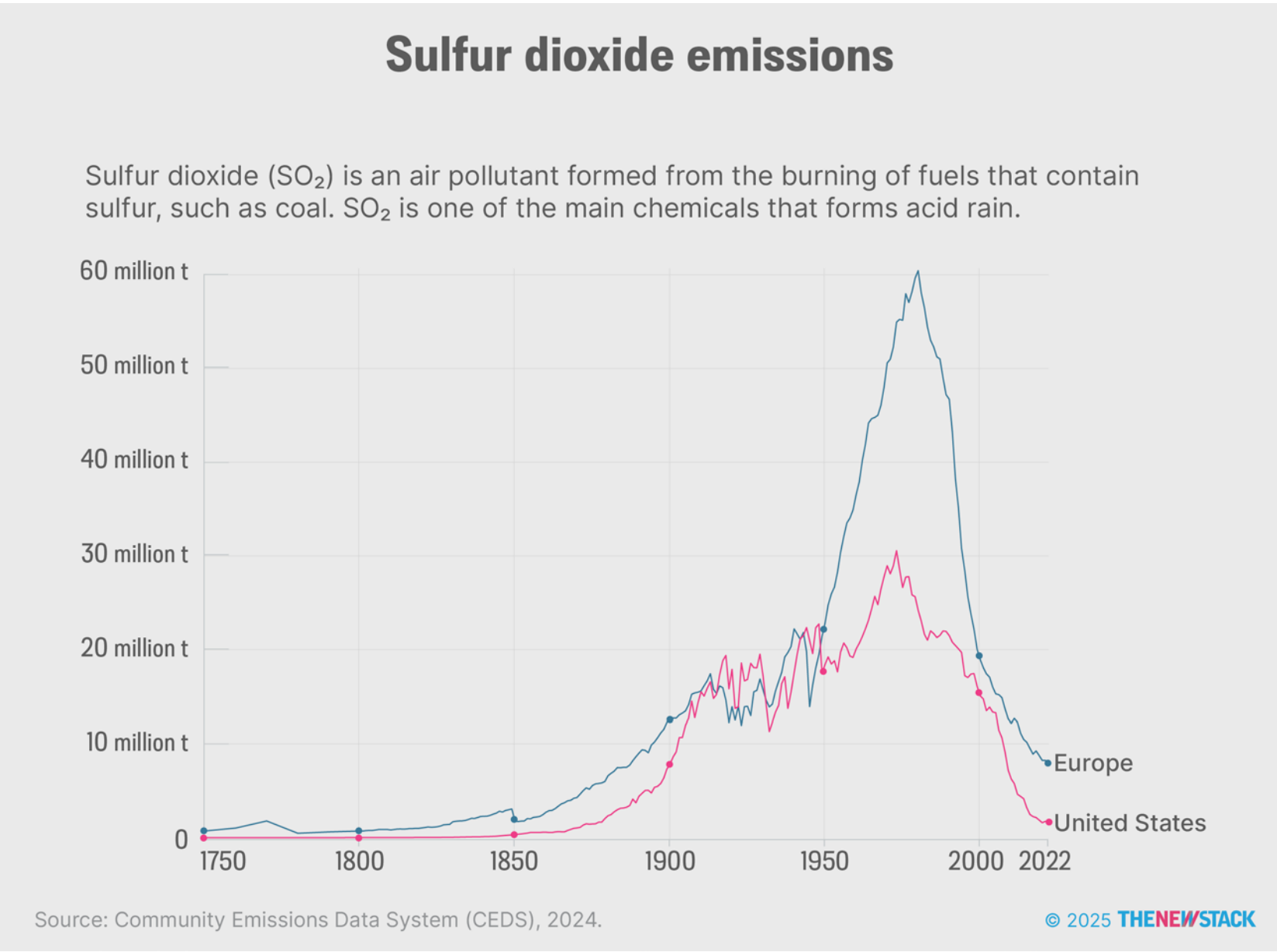


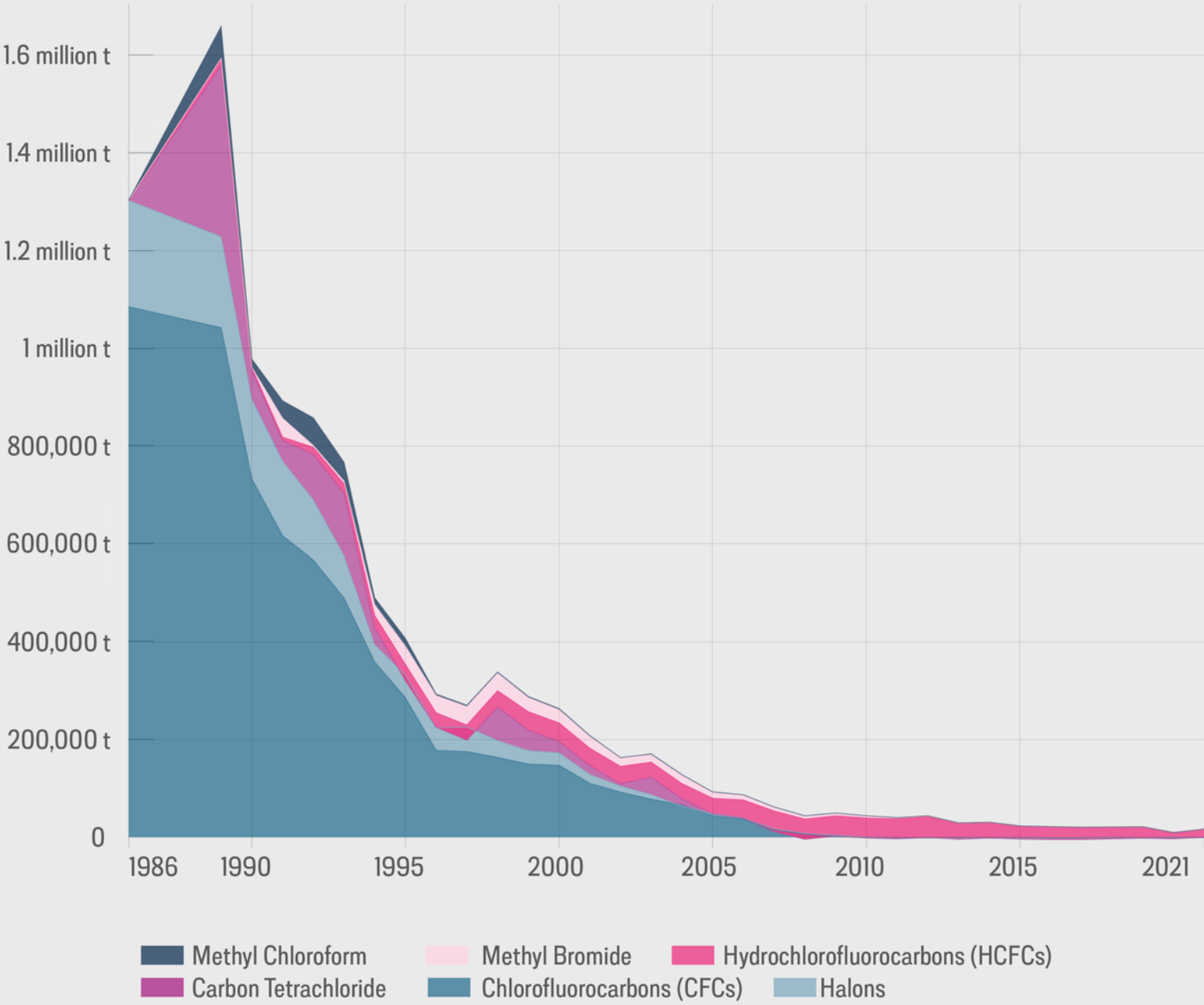
FIG 8.2 Sulfur dioxide emissions fell significantly after international agreements reduced major causes of acid rain.

International agreements, which placed emissions limits on power plants and started to reduce coal burning, were incredibly effective. In Europe, sulfur dioxide emissions fell by 84%, and in the U.S. by 90%.

Likewise, after countries signed the Montreal Protocol in 1987, emissions of ozone-depleting substances [fell by more than 99%:](#)

Emissions of ozone-depleting substances - World

Annual consumption of ozone-depleting substances. Emissions of each gas are given in ODP tons¹, which accounts for the quantity of gas emitted and how "strong" it is in terms of depleting ozone.



Note: In some years, gases can have negative consumption values. This occurs when countries destroy or export gases that were produced in previous years (e.g. stockpiles).

1. Ozone-depleting tons (ODP tons): Ozone-depleting tons measure the total potential of substances to deplete the ozone layer. Some substances that deplete the ozone layer are "stronger" than others, meaning one ton will cause greater damage than one ton of another. ODP tons are calculated by multiplying a substance's emissions in tons, by its "ozone-depleting potential". Ozone-depleting potential measures how much depletion a substance causes relative to CFC-11, which has a value of 1.0. If one ton of a gas caused twice the depletion of CFC-11, it would have a potential of 2.0.

Source: UN Environment Programme, 2023 © 2025 THE NEW STACK

FIG 8.2 Ozone-depleting substances fell significantly after the Montreal Protocol was signed in 1987.

Carbon Taxation

What other regulation would be helpful? In the wider context of decarbonizing the global economy, we almost certainly need some sort of carbon tax on everything we buy based on the gases emitted during production. In Hannah Richie's book "[Not the End of the World](#)," she says:

"I've asked many economists what we need to do to tackle climate change. Every single one has given me the same answer: put a price on carbon. It is, perhaps, the only thing that economists agree on."

The EU has taken steps toward this, announcing the world's first [Carbon Border Adjustment Mechanism](#) (CBAM), which aims to make all EU suppliers green. Currently in a transition period, it will be fully operational in January 2026.

The EU says that within nine years it will cover all goods and services coming into the EU, including those generated by the IT industry. While it applies only in the EU, it might follow the model established by GDPR, the European data protection legislation, where other countries have followed the EU's lead.

Water Usage

We've not talked about water use in this ebook because, as software developers, our direct impact here is low. However, data center operators do need to be mindful of a trade-off between energy and water use.

For warehouse-scale data centers, water-based cooling is the most common approach used to dissipate heat.

And among water-cooling solutions, evaporative cooling — in which water absorbs ambient heat through evaporation — is frequently both the cheapest and most energy efficient. It is generally [around 10% more energy efficient](#) than air cooling. However, water-based cooling can still put pressure on local water resources. A [Virginia Tech study](#) estimates that one-fifth of U.S. data centers (primarily located in Western states) draw their water from moderately to highly stressed watersheds.

The tech giants are starting to explore new, more water-efficient approaches. [Meta, for example, experimented](#) with shifting the minimum relative humidity at its data center in Los Lunas, N.M., from 20% to 13%, based on guidance from the American Society of Heating, Refrigerating, and Air-Conditioning Engineers. The nine-month pilot yielded water savings of 40%.

Data center operators are also taking steps to replenish their water consumption. Google, for example, has a target to replenish 120% of the freshwater volume it consumes on average by 2030, through initiatives including wetland restoration, rainwater harvesting, and land conservation.

Looking ahead, the water-energy trade-off in data center cooling will likely intensify as climate change strains water resources and increases cooling demands. While technical innovations like humidity adjustments and water recycling show promise, the industry may need to pursue more radical solutions — from waterless cooling technologies to strategic data center siting that considers watershed stress.

Software developers can support these efforts by designing applications that minimize computational intensity and choosing cloud providers with strong water stewardship practices.



CHAPTER 09

Final Words

One company that has applied much of the advice in this ebook is cosmetics giant L'Oréal. Employing 90,000 employees in more than 150 different countries, L'Oréal sold 7 billion products in 2023, with revenue of 41.2 billion euros. [Speaking at Google Cloud Next](#), in 2024, Antoine Castex, group data architect at L'Oréal, said that the company's digital services' carbon emissions were four-times greater than that from the factories that produce those 7 billion items.

In terms of sustainability, the cosmetics company started by moving its workloads to the cloud. L'Oréal has a multicloud strategy, and it exports the data from the Google Cloud Footprint tool and its other providers into a custom-built cloud sustainability dashboard. As much as it can, it uses a serverless-first approach to building software. L'Oréal's developers work using Google Cloud Workstations, with the lowest carbon region automatically configured to autoscale to zero.

Building sustainable software is both a technical and moral imperative for our industry. While the challenges we face are significant, we have the knowledge — and increasingly the tools and regulatory frameworks — needed to make meaningful progress. As developers, we can drive positive change through practical steps like

proper sizing, leveraging serverless architectures, employing demand shifting, and making full use of cloud capabilities.

But perhaps more importantly, we can help shape how our industry evolves by making sustainability a first-class concern in our architectural decisions, putting pressure on vendors to improve their practices, and supporting open source projects that prioritize efficiency.

The transition to sustainable software won't happen overnight, but by combining individual action with systemic change, we can help ensure that the digital infrastructure we're building today helps create the sustainable world we need tomorrow.

For more insight, refer to the further resources below.

Further Reading

In “[Building Green Software](#),” Anne Currie, Sarah Hsu, and Sara Bergman provide a more detailed introduction to the subject than could be covered here.

“[The Cloud Native Attitude](#),” by Anne Currie with Jamie Dobson, includes a chapter on sustainability and several case studies.

“[How To Avoid a Climate Disaster](#),” by Bill Gates, is an excellent introduction to the broader topic of climate change.

“[Not the End of the World](#),” by Hannah Richie, provides wonderful, optimistic, and meticulously researched insight into dealing with the climate disaster.

About the Author



[Charles Humble](#) is a freelance consultant, author, and podcaster.

A former software engineer, architect, and CTO, he has worked as a senior leader and executive of both technology and content groups.

He was InfoQ's editor-in-chief from 2014 to 2020, and was chief editor for Container Solutions from 2020 to 2023.

He writes regularly for The New Stack and other publications, is a highly experienced content strategist, and has spoken at multiple international conferences including Devvxx, GOTO, and QCon.

His primary areas of interest are how we build software better, including sustainability and ethics, cloud computing, remote working, diversity and inclusion, and inspiring the next generation of developers.

Charles is also a keyboard player, and half of the ambient techno band [Twofish](#).

Disclosure

The following companies mentioned in this ebook are also sponsors of The New Stack: Amazon Web Services, Red Hat, and VMware by Broadcom.

TNS owner Insight Partners also is an investor in Honeycomb, which is mentioned in this ebook.

